Faculty of Engineering and Materials Sciences
German University in Cairo



Design and Implementation of a Smart Cleaning Robot

Bachelor Thesis

Author:	Andrew Khalil Samuel Abdelmalak
Supervisors:	Prof. Dr. Walid Atef Hafez Omran
Co-Supervisor:	Dr. Hisham Mostafa El Sherif
Reviewer:	Dr. M. A. Moustafa Hassan
Submission Date:	22 June, 2025

This is to certify that:

(i) the thesis comprises only my original work toward the Bachelor Degree

(ii) due acknowledgement has been made in the text to all other material used

---

Andrew Khalil Samuel Abdelmalak
22 June, 2025

Acknowledgements

I would first like to thank God for granting me the strength, focus, and perseverance to complete this work.

I am deeply grateful to my supervisor, Prof. Dr. Walid Atef Hafez Omran, whose expertise in electrical engineering, patient guidance, and enthusiastic encouragement were invaluable throughout every stage of this thesis.

I would also like to express my sincere appreciation to my co-supervisor, Dr. Hisham Mostafa El Sherif, whose mechanical-engineering perspective and constructive feedback greatly enriched both the research and the writing of this work.

Finally, I wish to thank my parents and family for their unwavering support, patience, and constant encouragement.

Andrew Khalil Samuel Abdelmalak
22 June, 2025

Dedication

To my parents,
for their endless support, patience,
and unwavering belief in me.

Abstract

Household cleaning is a repetitive and time-consuming task, often detracting from other valuable activities in increasingly busy modern lifestyles. This project addresses the persistent need for automation in domestic environments by focusing on the critical area of floor cleaning, aiming to alleviate this burden. The primary objective is the comprehensive design, development, and implementation of a "Smart Cleaning Robot" prototype, conceived to simplify cleaning routines and significantly reduce direct human effort and interaction. This thesis meticulously details the iterative development journey of this prototype, from conceptualization through multiple design refinements to final assembly.

The robot's navigation has been engineered into a hybrid feed-forward and feedback control system, utilizing a predictive model to proactively manage motor commands while using a Proportional-Integral-Derivative architecture to correct for real-world deviations. This enables the robot to navigate more methodically using a coverage pattern of moving forward until an obstacle is detected and then executing a wide, corrective turn. This system allows it to manage its movements with greater precision and perform autonomous recovery maneuvers. The robot's core functionality relies on an integrated vacuum system, powered by a PS3 Slim fan. Central control is managed by a Raspberry Pi Pico W microcontroller, which processes data from an ultrasonic distance sensor for real-time obstacle detection and from the encoders for motion control. Mobility is achieved through two independently controlled Direct Current motors, and the entire system is powered by a rechargeable Lithium-ion battery pack.

The project's methodology encompassed a structured, iterative design process that included detailed mechanical modeling using SolidWorks software, the fabrication of a custom 3D-printed chassis, and the design and implementation of a dedicated Printed Circuit Board. The prototype achieves its core functionality at approximately 60–90% lower cost than entry-level commercial robots, with a total component cost of approximately $80 USD. Testing demonstrated stable BLE remote control, improved navigation precision through PID control, and an operational battery runtime exceeding 60 minutes. While the prototype successfully demonstrates effective vacuuming and responsive remote control, the core of the work lies in the implementation and analysis of the advanced control system. This thesis candidly highlights the successes in implementing this system, the remaining precision challenges inherent to the chosen hardware, and the key recommendation of exploring stepper motors for future work, thereby contributing to the accessibility of domestic robotic assistance.

Table of Contents

Acknowledgements **iii**

Abstract **v**

Table of Contents **vi**

List of Figures **viii**

List of Tables **x**

List of Abbreviations **xi**

List of Symbols **xiii**

1 Introduction **1**
1.1 Motivation 1
1.2 Aim of the project 2
1.3 Thesis Structure 2

2 Theoretical Background and Literature Review **4**
2.1 Overview of Cleaning Robot Technologies 4
2.2 Analysis of Existing Designs 10
2.2.1 Non-Commercial / Custom-Built Designs 10
2.2.2 Commercial Cleaning Robot Designs 15
2.3 Key Takeaways and Problem Statement 18

3 Mechanical Design and Assembly **20**
3.1 Introduction 20
3.2 Mechanical Components Summary 20
3.3 Detailed Mechanical Design 21
3.3.1 Chassis: Structure and Material 21
3.3.2 Vacuum System Design 23
3.3.3 Drive System Design 26
3.3.4 Main Robot Lid and Component Fastening 27
3.4 Mechanical Assembly Implementation 27

4 Electrical and Control System Design and Testing **29**
4.1 Introduction . . . 29
4.2 Electrical Components Summary . . . 29
4.3 System Architecture . . . 33
4.4 Electrical Design and Integration . . . 33
4.4.1 Power Distribution . . . 33
4.4.2 Component Interconnections and Wiring . . . 34
4.4.3 Custom PCB Design and Fabrication . . . 37
4.5 Control System Implementation (Software) . . . 37
4.5.1 Software Architecture . . . 37
4.5.2 Evolution of the Control Strategy . . . 40
4.5.3 BLE Remote Control . . . 43
4.5.4 Software Deployment . . . 44
4.6 Performance Evaluation and Results . . . 45
4.6.1 Functional Verification and Observations . . . 45
4.6.2 Summary . . . 46

5 Conclusion and Recommendations for Future Work **47**
5.1 Conclusion . . . 47
5.1.1 Cost Analysis . . . 47
5.2 Limitations . . . 48
5.3 Lessons Learned . . . 49
5.4 Recommendations for Future Work . . . 49

A Bill of Materials and Cost Analysis **54**

List of Figures

2.1 Illustration of the Boustrophedon coverage path pattern. 5
2.2 A comparison of a random navigation path versus a systematic path enabled by SLAM (Concept from [3]). 6
2.3 An example of a 3D point-cloud map of an environment generated using SLAM (Concept from [4]). 6
2.4 Illustration of the HC-SR04 ultrasonic sensor's working principle (Concept from [8]). 8
2.5 Thumbnail of The Ultimate Floor Cleaning Robot (Version 2.0) [11]. . . . 11
2.6 Thumbnail of Arduino based Smart Vacuum Cleaner Robot [12]. 12
2.7 Thumbnail of Self-Navigating Vacuum Cleaner [13]. 12
2.8 Thumbnail of The Amazing Engineering behind Cleaning Robots [14]. . . 13
2.9 Photo of iRobot Roomba 980 [15]. 15
2.10 Photo of Xiaomi Robot Vacuum X20 Plus [17]. 17

3.1 SolidWorks CAD models illustrating the Smart Cleaning Robot's chassis and assembled form, with annotations to highlight key features. 22
3.2 Bottom view photograph of the assembled Smart Cleaning Robot, showing the caster wheel, drive wheels, and vacuum intake nozzle. 24
3.3 SolidWorks CAD design of the Debris Collection Chamber. 25
3.4 Photograph of the 3D printed Debris Collection Chamber with its removable filter lid. 26
3.5 Diagram illustrating the principle of operation for the optical wheel encoder. As the encoder wheel (slotted disc) rotates with the main robot wheel, its slots alternately block and pass the light beam from the source to the sensor, generating a digital pulse train that corresponds to the distance traveled. 27
3.6 Photographs of the 3D printed and assembled robot prototype. 28

4.1 High-level control flowchart of the autonomous cleaning mode, showing user-initiated activation, forward motion with hybrid PID control, obstacle avoidance, and stall recovery paths. 34
4.2 Raspberry Pi Pico W pinout diagram. Source: [9]. 35
4.3 Conceptual validation stages: (a) Proteus simulation of the robot's basic obstacle avoidance logic using an Arduino model, and (b) photograph of the breadboard prototype showing key component interconnections for physical validation. 37

4.4 Internal view of the assembled robot showing component wiring and integration with the custom PCB. 38
4.5 KiCad schematic diagram for the custom Printed Circuit Board. 39
4.6 KiCad PCB layout design (2D view). 39
4.7 3D rendering of the populated PCB design from KiCad. 40
4.8 Block diagram of the autonomous obstacle-avoidance state machine showing the robot's operational states and transitions. 41
4.9 Example user interface of the NRF Toolbox application configured for controlling the Smart Cleaning Robot via BLE UART. 44
4.10 Screenshot of Thonny IDE showing the MicroPython control script deployed to the Raspberry Pi Pico W. 44

List of Tables

3.1 Mechanical Components List 20

4.1 Electrical Components List 29
4.2 Performance Test Results Summary. 46

5.1 Cost Comparison: Prototype vs. Commercial Robots. 48

A.1 Comprehensive Bill of Materials 54

List of Abbreviations

ADC: Analog-to-Digital Converter.

BLE: Bluetooth Low Energy.

CAD: Computer-Aided Design.

DC: Direct Current.

FDM: Fused Deposition Modeling.

FF: Feed-Forward.

GPIO: General Purpose Input/Output.

HC-SR04: Ultrasonic Sensor Model.

IDE: Integrated Development Environment.

IMU: Inertial Measurement Unit.

IR: Infrared.

L298N: H-Bridge Motor Driver Model.

LDS: Laser Direct Structure.

LED: Light Emitting Diode.

LiDAR: Light Detection and Ranging.

Li-ion: Lithium-ion.

PCB: Printed Circuit Board.

PID: Proportional-Integral-Derivative.

PLA: Polylactic Acid.

PWM: Pulse Width Modulation.

ROS: Robot Operating System.

RPLidar: Rotating Light Detection and Ranging sensor model.

RPM: Revolutions Per Minute.

SBC: Single-Board Computer.

SLAM: Simultaneous Localization and Mapping.

UART: Universal Asynchronous Receiver-Transmitter.

VBUS: USB Voltage Bus.

vSLAM: Visual Simultaneous Localization and Mapping.

List of Symbols

V: Voltage (*Volts*)

I: Current (*Amperes*)

P: Power (*Watts*)

R: Resistance (*Ohms*)

f: Frequency (*Hertz*)

T: Time Period (*Seconds*)

D: Duty Cycle

θ**:** Angle (*Degrees*)

L: Length (*cm/m*)

W: Width (*cm/m*)

H: Height (*cm/m*)

m: Mass (*kg/g*)

gf.cm: Gram-force centimeter (*Torque unit*)

Chapter 1

Introduction

1.1 Motivation

Keeping our homes and workplaces clean is important for a healthy and pleasant life. But cleaning floors by hand often takes a lot of time, is boring, and can be hard physical work. In today's busy world, people have many things to do. Finding enough time and energy for regular cleaning can be tough, especially in big homes or busy business places. New technologies, especially in robotics, offer a good solution. Cleaning robots, which were once just an idea, are now helpful tools. They make cleaning easier, do a consistent job, and most importantly, save people's time. This saved time can be used for other important tasks, learning new things, or relaxing. The development of this technology has not only changed how we do chores but has also created new business opportunities and markets. For example, companies like iRobot, with its famous Roomba, were pioneers. They showed that many people wanted these new cleaning robots. After that, big home appliance companies like Samsung, LG, Dyson, and SharkNinja, and robot companies like Ecovacs and Roborock, also started making their own cleaning robots. They saw it as an important and growing part of the smart home market. This shows how automated cleaning has made a big impact, turning a new idea into a large global business. This project looks at cleaning in Egyptian homes. In Egypt, many floors are made of tile or marble. Also, some areas can have a lot of dust. These conditions make automated cleaning robots very useful. Many advanced cleaning robots are sold worldwide, but they can be very expensive for some people. Because of this, and because people might want robots that fit their specific needs, there's a chance to create simpler, effective robots that can be made or put together locally. So, the main reason for this bachelor thesis is to design and build a smart cleaning robot that works well, is practical, and doesn't cost too much. The goal is to make a robot that greatly cuts down on the handwork needed for common floor cleaning, making homes more efficient and life easier. This project tries to find a good balance between useful features and cost, offering a helpful step in home robotics.

1.2 Aim of the project

The primary technical aim of this work is to design and implement a functional prototype of a smart cleaning robot. This robot is intended to automate common floor cleaning tasks, thereby reducing manual effort and enhancing convenience within a typical household environment. The project focuses on delivering a practical and cost-effective solution that balances essential functionalities with simplicity of operation. The specific tasks to achieve this aim involve the implementation of the following core robot functions:

- **Automated Vacuum Cleaning:** The robot will be equipped with a vacuuming system capable of collecting common household dust and light debris from floor surfaces. This function is central to its cleaning purpose.
- **Enhanced Autonomous Navigation:** The robot will implement a hybrid feedforward and PID control system using wheel encoders. The aim is to significantly improve movement precision, enabling the robot to more reliably execute a systematic room coverage pattern based on its sensor feedback. The system will also include autonomous stall detection and a recovery sequence.
- **Remote Control Operation:** The robot will feature a BLE remote-control interface, enabling users to manually guide its movements. This is enhanced by PID control to maintain a straight path during forward motion and an automatic obstacle-stop safety feature.
- **User Feedback System:** The robot will incorporate basic visual indicators (e.g., an LED) to provide feedback to the user about its current operational status.

The successful implementation and qualitative verification of these functions will demonstrate the achievement of the project's technical aim. This project aims to deliver a foundational smart cleaning robot prototype, with the understanding that highly advanced features like sophisticated mapping (SLAM) are outside the current scope.

1.3 Thesis Structure

This thesis is organized into five chapters. Each chapter discusses a different part of the project, providing a clear and logical progression from initial concepts to final outcomes and future possibilities:

- **Chapter 1: Introduction (This Chapter)** This chapter provides the background for the project. It explains the motivation for developing a smart cleaning robot, outlines the project's objectives, and details the organization of this document.
- **Chapter 2: Theoretical Background and Literature Review** This chapter delves into the technologies, design principles, and prior research relevant to autonomous and semi-autonomous cleaning robots. It examines various non-commercial and commercial robot designs, focusing on their navigation strategies, sensing capabilities, cleaning mechanisms, and control systems. It also reviews specific path planning techniques like the boustrophedon pattern.

- **Chapter 3: Mechanical Design and Assembly** This chapter focuses on the physical construction of the robot. It details the selection of mechanical components, the SolidWorks-based design of the chassis and custom parts, the selection of materials, the mechanical assembly process, and the specifics of the vacuum system's mechanical integration.
- **Chapter 4: Electrical and Control System Design and Testing** This chapter describes the electronic "brains and nerves" of the robot. It covers the selection of electrical components, the overall system architecture centered on the Raspberry Pi Pico W, the detailed electrical design and integration including the custom PCB, and the embedded software development for control. Finally, it presents a qualitative performance evaluation of the assembled prototype.
- **Chapter 5: Conclusion and Recommendations for Future Work** This chapter summarizes the project's main achievements and findings. It reflects on the extent to which the initial goals were met, discusses the identified limitations, and proposes recommendations for future research and potential improvements to enhance the robot's functionality and performance.

This structured approach aims to provide a comprehensive account of this project from its inception to its current state.

Chapter 2

Theoretical Background and Literature Review

This chapter gives a broad look at the main technologies, common design methods, and important past work related to cleaning robots that work by themselves or with some human help. By looking at existing research, non-commercial projects, and store-bought robots, this review will help create a strong base of knowledge for the design choices made in this thesis. The chapter will explore key areas like how robots move around, how they detect and avoid objects, different ways they clean, how their control systems work, and how they manage power. It will also touch upon specific coverage patterns relevant to systematic cleaning.

2.1 Overview of Cleaning Robot Technologies

Automated cleaning robots have changed a lot over the years. They started as simple machines and have become much smarter. This happened because of better sensors, more powerful computer chips, and improved software that help them work more independently and effectively.

Robotic Navigation and Environmental Mapping Strategies

A robot's ability to navigate its environment effectively is fundamental. Various techniques exist, ranging in complexity and the level of autonomy they provide:

- **Basic Reactive Strategies:** These are the simplest forms of navigation.
  - *Random Walk:* The robot moves until an obstacle is detected (e.g., by a bump sensor), then changes direction, often randomly. This is low-cost but inefficient for coverage.
  - *Predefined Path Following and Algorithmic Patterns:* Robots may follow a set path or execute geometric patterns (spirals, wall-following). These offer more structure than random walks and can use simple sensors like line-followers or odometry (wheel tracking) with basic proximity sensors for obstacle avoidance. They are best for known, uncluttered spaces. A fundamental component

for odometry is the wheel encoder, a sensor that measures the rotation of the wheels. By counting the rotational "pulses," the robot can calculate the distance it has traveled with high precision, enabling more reliable execution of systematic coverage patterns.

- **Systematic Coverage Path Planning:** For more thorough cleaning, robots aim to cover an entire accessible area without unnecessary repetition. A common approach for this is the boustrophedon pattern (also known as an ox-plowing or lawnmower pattern) [1]. This involves moving back and forth in parallel lines, much like mowing a lawn, to ensure complete coverage of a rectangular area. Figure 2.1 illustrates this pattern. Achieving this reliably requires accurate localization and control, often using wheel encoders or more advanced sensing. While this systematic pattern represents a common and efficient goal in robotics, the prototype developed in this thesis implements a simpler coverage strategy based on direct sensor feedback as a practical step toward this ideal, working within the precision limits of the available hardware.

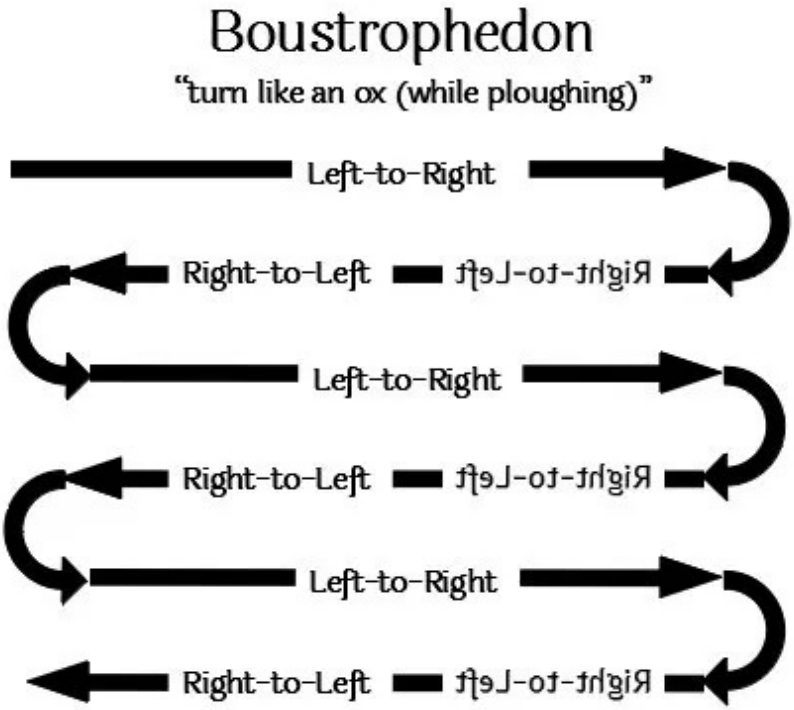


Figure 2.1: Illustration of the Boustrophedon coverage path pattern.

- **Advanced Mapping and Localization:** SLAM represents a major step in autonomy [2], allowing a robot to build a map of an unknown area while simultaneously tracking its position. This results in a much more efficient and systematic cleaning path compared to random navigation (Figure 2.2).
  This is computationally intensive and relies on rich sensor data (like from LiDAR or cameras) to create a spatial awareness map, which can be a 2D grid or a more complex 3D point cloud, as shown in Figure 2.3.

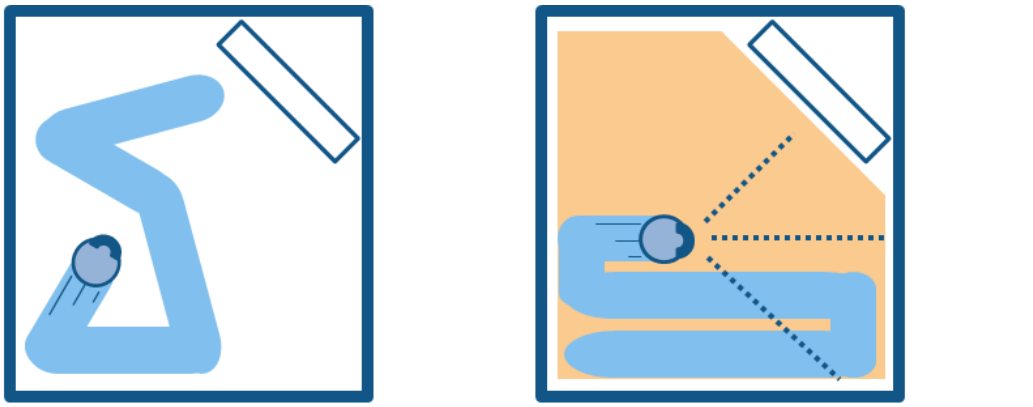


Figure 2.2: A comparison of a random navigation path versus a systematic path enabled by SLAM (Concept from [3]).

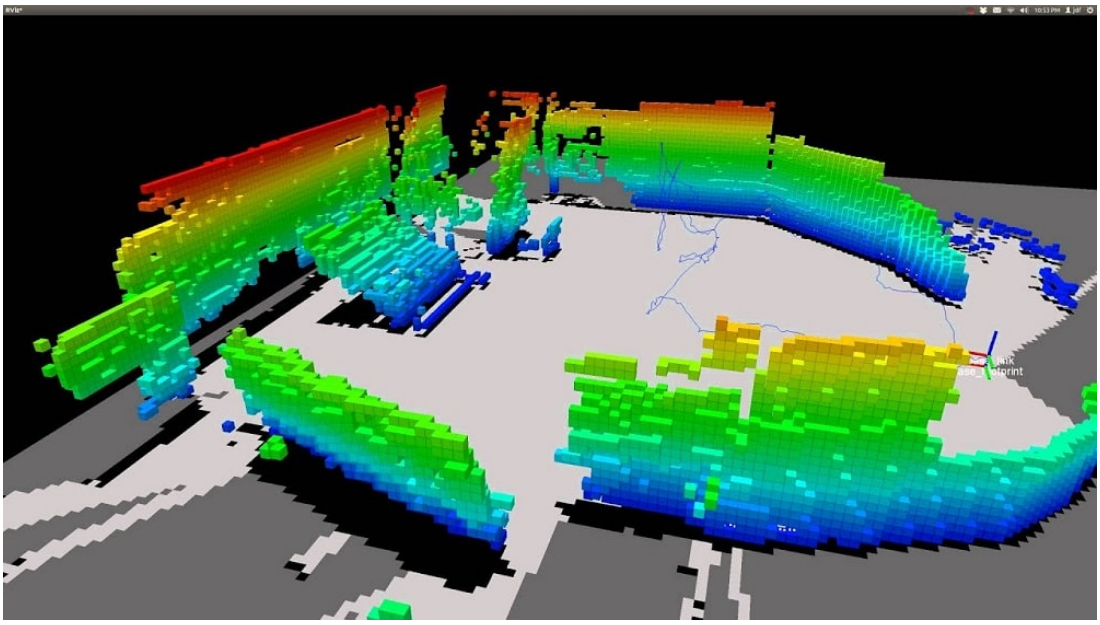


Figure 2.3: An example of a 3D point-cloud map of an environment generated using SLAM (Concept from [4]).

- **PID Feedback Control:** To achieve the precision needed for systematic patterns, a feedback control loop is essential. The Proportional-Integral-Derivative (PID) controller is a ubiquitous and robust method used for this purpose. It continuously calculates an error value between a desired setpoint and a measured process variable and applies a correction based on proportional, integral, and derivative terms. A detailed explanation of this control method, as it applies to this project, is provided in the following section.

Principles of PID Feedback Control

To move beyond simple reactive strategies, a feedback control system is necessary to allow the robot to correct its own errors in real-time. The most common and robust method for this is the **Proportional-Integral-Derivative (PID)** controller [5]. A PID controller works by calculating an "error" value—the difference between a desired setpoint (e.g., a target wheel speed) and the current measured value—and then calculating a corrective action to apply to the system (e.g., motor power). The power of the PID controller comes from its three distinct terms, each addressing a different aspect of the error.

- **The Proportional (P) Term: Reacting to the Present**
  The Proportional term provides a response that is directly proportional to the current error. If the error is large, the corrective action is large; if the error is small, the correction is small. For our robot driving straight, the error could be the difference in speed between the two wheels. The P-term works to make this error zero. However, relying on the P-term alone can lead to instability. A high proportional gain (the scaling factor, K_p) can cause the system to **overshoot** the target, oscillating back and forth around the setpoint.

- **The Derivative (D) Term: Predicting the Future**
  The Derivative term works to counteract this overshoot. It looks at the **rate of change** of the error. If the error is decreasing rapidly (meaning the robot is quickly approaching its target), the D-term will apply a damping force to slow the correction down, preventing overshoot. It essentially acts as a brake, adding resistance against rapid changes and making the system's response smoother. Properly tuning the derivative gain (K_d) leads to a "**critically damped**" system that approaches the target quickly but without oscillation.

- **The Integral (I) Term: Correcting for the Past**
  The Proportional and Derivative terms can effectively correct for immediate errors, but they can struggle with persistent, **steady-state errors**. These can be caused by systemic biases, such as one motor being slightly weaker than the other or uneven friction on the wheels. The Integral term solves this by summing up the cumulative error over time. If a small error persists, the integral sum grows larger and larger, eventually building up enough corrective force to overcome the systemic bias and drive the error to zero. The integral gain (K_i) must be tuned carefully; if it is too high, it can cause its own oscillations and instability.

The final control output is the sum of these three terms. The process of "tuning" a PID controller involves finding the optimal balance for the three gains (K_p, K_i, K_d) to achieve a fast, stable, and accurate response, a process central to the work detailed in Chapter 4.

Feed-Forward Control: A Proactive Approach

While PID control is excellent at reacting to errors, it must wait for an error to occur before it can act. An alternative and complementary strategy is **Feed-Forward (FF)** control, which acts proactively to prevent errors from happening in the first place [6].

A simple analogy is driving a car up a hill. A purely reactive feedback controller would only press the accelerator *after* it detects the car's speed has dropped. In contrast, a feed-forward controller, knowing a hill is coming, presses the accelerator *before* the speed drops, anticipating the required effort [6]. The hybrid system developed for this thesis combines both strengths: a feed-forward model provides the primary motor command based on a desired speed, while the PID controller provides fine-tuning adjustments to correct for unmodeled disturbances, such as changes in friction or minor wheel slippage.

It is important to distinguish between the navigation or mapping *technique* (e.g., SLAM, boustrophedon decomposition) and the *sensors* that provide data for these techniques.

Obstacle Detection and Avoidance

To operate safely, robots need sensors to perceive and react to obstacles. Common types include:

- **Ultrasonic Sensors:** Emit sound waves and measure the echo time to determine distance. They are cost-effective for detecting larger obstacles but can struggle with soft, narrow, or angled surfaces. They operate by emitting an ultrasonic wave and measuring the time it takes for the echo to return, as illustrated in Figure 2.4. The HC-SR04 is a common example [7].

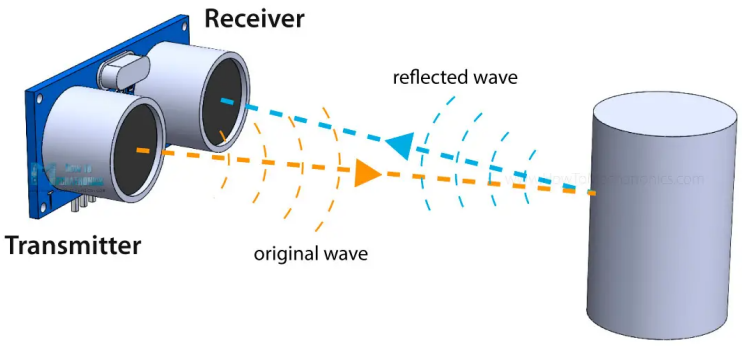


Figure 2.4: Illustration of the HC-SR04 ultrasonic sensor's working principle (Concept from [8]).

- **Infrared Proximity and Cliff Sensors:** Used for short-range detection and crucially as "cliff sensors" (typically under the robot) to detect drops like stairs by measuring reflected IR light.

- **Optical Limit Switches (Bump Sensors):** Physical contact sensors, usually in the robot's bumper, that activate upon collision, providing a definitive, albeit reactive, obstacle signal.
- **3D Depth Sensors:** Provide a richer 3D understanding of surroundings. Examples include stereo cameras (using two cameras for depth via parallax) and structured light sensors (projecting light patterns). They help detect complex or low-lying obstacles.
- **LiDAR for Obstacle Detection:** While key for SLAM, LiDAR (e.g., RPLidar models) also directly provides precise distance measurements in a point cloud, useful for immediate obstacle detection.
- **Cameras for Obstacle Detection:** Similarly, cameras, vital for vSLAM, can also be used directly for obstacle detection via computer vision algorithms that identify objects in the robot's path.

Cleaning Mechanisms

Robots use different tools to clean:

- **Vacuum Suction:** This is the most common way. A fan or motor creates suction to pull dust and dirt into a container. How well this works depends on the motor's power, how the airflow is designed, and the shape of the vacuum nozzle.
- **Rotating Brushes:** These are often used with vacuuming. Main brushes (roller brushes) help loosen dirt from carpets or sweep hard floors. Side brushes stick out from the robot's sides to sweep dirt from edges and corners into the main cleaning area.
- **Mopping Systems:** Some robots can also mop. Simple ones might just drag a wet or dry cloth. More advanced ones have water tanks, control how much water is used, and sometimes have mop pads that vibrate or scrub.

Control Systems

The part of the robot that controls everything can be different:

- **Microcontroller-Based Logic:** Simpler robots often use small computer chips called microcontrollers (like Arduino or ESP32, or the Raspberry Pi Pico W [9] in this project). These are programmed with specific instructions for moving, using sensors, and cleaning. They are cheap and good for basic tasks.
- **Single-Board Computers:** More powerful options like the Raspberry Pi (the full version, not Pico) can run full computer operating systems (like Linux). This allows them to use more complex software, making them suitable for computationally intensive tasks like SLAM or running frameworks like ROS.

- **Robot Operating System:** ROS is a set of software tools and libraries for building robot applications [10]. It helps with tasks like navigation, understanding sensor data, and controlling robot parts. Using ROS usually needs a more powerful processor, like those in SBCs.

Power Management

Robots that move on their own need a battery:

- **Rechargeable Batteries:** These are standard. Older robots used NiCd or NiMH batteries.
- **Lithium-ion Batteries:** These are now the most popular because they store more energy in a smaller and lighter battery, last longer, and don't have the "memory effect" that older batteries did.
- **Automatic Recharging:** Many store-bought robots can automatically go back to a charging station when their battery is low. Some can even continue cleaning from where they stopped after recharging.

How these different technologies are used together determines what a cleaning robot can do, how well it works, and how much it costs. This project will choose carefully from these options to meet its specific goals.

2.2 Analysis of Existing Designs

To help design the prototype for this project, I looked at several existing cleaning robot designs. This included simple non-commercial or custom-built projects, which are often easy and cheap to build, and more advanced commercial robots, which show high-end features and performance.

2.2.1 Non-Commercial / Custom-Built Designs

Non-commercial projects give good ideas on how to get basic functions working with few resources and common parts. They often show smart solutions and prove that core robot tasks can be done using common microcontrollers and sensors.

Design 1: Ultimate Floor Cleaning Robot (Version 2.0) [11]

This project by Saiyam Agrawal is a robot that can mop and move around on its own in a basic way.

Key Features & Analysis:

- **Navigation:** It uses three HC-SR04 sensors (front, left, right) to see obstacles. An Arduino Nano controls it. The robot moves backward and turns based on what the sensors see. Using three sensors helps it see more around it than just one sensor. However, the creator mentions that its movement plan can have trouble in tricky spots like corners. This shows that simply reacting to what's nearby without a map can be difficult.



Figure 2.5: Thumbnail of The Ultimate Floor Cleaning Robot (Version 2.0) [11].

- **Cleaning:** It has a fixed mop. Water drips onto the mop from a bottle through a medical drip valve. This is a simple and cheap way to mop without needing motors for the mop. But, a problem noted is that the mop gets heavy when wet, which can make the wheels slip and affect how the robot moves.
- **Control:** It can be controlled manually with a smartphone app using Bluetooth (HC-05 module). This makes it user-friendly.
- **Power:** It uses three 18650 Li-ion batteries connected together for 12V power. This shows that Li-ion batteries are chosen because they are light.
- **Lessons for this Thesis:** This design shows it's possible to use several sensors and a cleaning tool with a simple microcontroller like an Arduino. The problems with complex navigation and the wet mop's weight affecting movement are important things to think about for the proposed robot's design and how weight is spread.

Design 2: Arduino-Based Smart Vacuum Cleaner Robot [12]

This project by "Rizwan's Ideas" is about making a vacuuming robot mostly from recycled items.

Key Features & Analysis:

- **Building Materials:** It focuses on saving money by using a plastic bottle and cardboard for the robot's body.
- **Navigation:** An Arduino Uno controls four DC motors using a motor shield. One HC-SR04 ultrasonic sensor is put on a servo motor. This lets the sensor turn and scan for obstacles. Scanning can help see a wider area than a fixed sensor, but it makes the robot more complex mechanically and takes time to scan.
- **Cleaning:** The vacuum system is very basic, using a small 6V fan motor inside the plastic bottle. The creator says the suction is not very strong. This is a common issue in very cheap custom-built vacuum designs that don't use special high-speed fans.



Figure 2.6: Thumbnail of Arduino based Smart Vacuum Cleaner Robot [12].

- **Lessons for this Thesis:** This design shows how to be creative with materials. The sensor on a servo is an interesting way to make one sensor see more, but it has its own difficulties. The weak suction shows that choosing a good fan and designing good airflow is important for vacuuming. Using four motors might be for better movement or just because those parts were available.

Design 3: Self-Navigating Vacuum Cleaner (Video by SciNix) [13]

This project by SciNix shows a more advanced custom-built robot that uses the ROS.

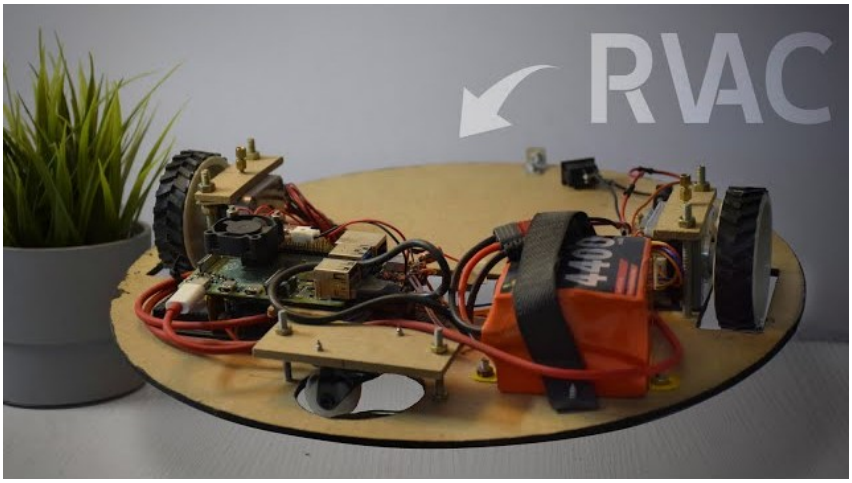


Figure 2.7: Thumbnail of Self-Navigating Vacuum Cleaner [13].

Key Features & Analysis:

- **Control System:** It uses a Raspberry Pi 4 to run the ROS Navigation Stack. This software handles moving around on its own and planning paths based on maps made beforehand. An Arduino Mega is used as a simpler controller for the stepper motors.

- **Movement:** It uses stepper motors for very precise movement. This might mean it doesn't need wheel encoders if the steps are counted correctly, but it could have problems with wheels slipping if not handled well.
- **Navigation:** Using ROS is a big step up in software difficulty compared to typical Arduino projects. It allows for much smarter path planning (for example, using global and local planners to decide where to go).
- **Lessons for this Thesis:** This design shows a way to make a custom-built robot much more autonomous. Even though this thesis's project uses a Raspberry Pi Pico W (which is a microcontroller, not a full computer like the Pi 4) and doesn't plan to use ROS at first, the SciNix project shows that more powerful processors and software can greatly improve how a robot moves around. It reminds us to design the current robot so it can be upgraded in the future.

Design 4: Advanced Cleaning Robot Analysis (Video by Sabin Civil Engineering) [14]

This YouTube video provides an overview of the technologies and evolutionary stages of advanced cleaning robots, rather than detailing a single custom-built project. It discusses how these robots integrate various components and intelligence levels to achieve autonomous cleaning.

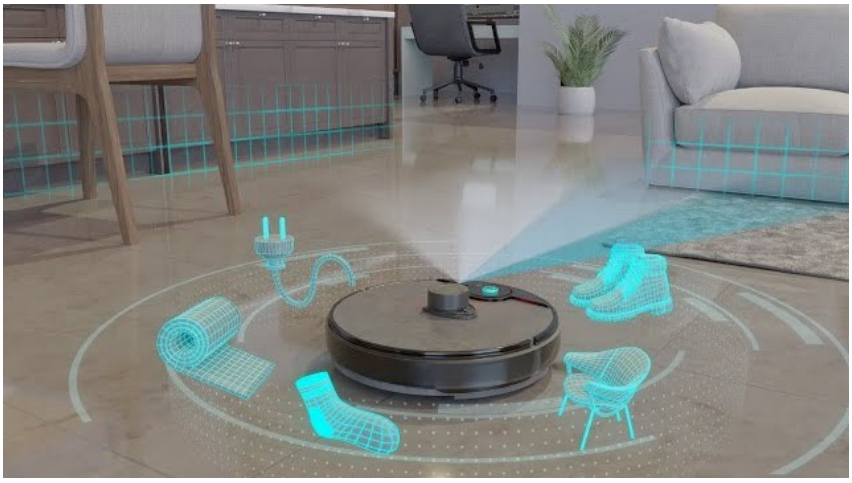


Figure 2.8: Thumbnail of The Amazing Engineering behind Cleaning Robots [14].

Key Ideas Discussed & Analysis:

- **Core Cleaning and Locomotion:** The foundation includes a vacuum pump for debris collection, driven by tractor wheels and a caster wheel for movement. To enhance cleaning width beyond the vacuum pump's direct opening, side brushes and a roller brush are employed to guide debris towards the suction area, improving efficiency, especially in corners. Some designs incorporate mopping attachments with controlled water flow for removing tough stains simultaneously with vacuuming.

- **Evolution of Obstacle Handling and Navigation:**
  - **Initial Reactive Obstacle Detection:** To address the issue of robots getting stuck, optical limit switches (bump sensors) are added to the front bumper. When an object is hit, these switches signal a smart controller.
  - **Basic Turning Maneuvers:** The controller then manages individual wheel speeds, enabling the robot to turn (e.g., by rotating wheels in opposite directions) after a collision. However, this random path is inefficient.
  - **Improving Path Efficiency:** For more systematic coverage, robots need to make accurate turns (e.g., right angles) and control travel distance to avoid excessive overlap. This leads to the integration of a gyroscope sensor for precise angular turns and wheel encoders to measure distance by counting wheel rotations.
  - **Advanced Obstacle Differentiation and Mapping:** A limitation of simpler sensor setups (bump, gyro, encoders) is the inability to differentiate between types of obstacles (e.g., a wall versus a chair leg), potentially leaving areas uncleaned. The introduction of a LiDAR sensor addresses this by emitting lasers and creating a 2D map of the room as it rotates. This map allows the robot to distinguish objects and follow more efficient cleaning paths.
  - **Cliff and Drop Detection:** Standard 2D LiDAR may not detect drops like staircases. To prevent falls, optical proximity sensors (typically IR emitters and detectors) are placed at the robot's edge. These measure reflected light, and a sudden loss of reflection signals a drop, prompting the controller to stop and redirect the robot.
  - **Detecting Small or Low-Lying Obstacles:** 2D LiDAR can also miss small or flat obstacles on the floor, such as wires, clothes, or pet waste. To address this, a camera, often combined with an inbuilt 3D depth sensor, is added to the front. This system maps the immediate frontal region in 3D, enabling precise detection and avoidance of such objects.
  - **Sensor Fusion:** The video highlights the Sensor Fusion technique, which involves combining data from multiple sensor types (e.g., LiDAR and camera systems) to achieve more accurate and robust environmental perception and cleaning paths.
- **User Interaction and Convenience Features:** Advanced robots often include mobile application integration for features like setting go/no-go zones, scheduling cleaning sessions, and targeting specific areas for cleaning.
- **Physical Capabilities:** The ability to climb small heights, such as door ramps (up to approximately 2 cm), is noted as a physical capability of some robots.
- **Power Management:** Automatic recharging, where the robot autonomously returns to its charging dock when the battery is low, is a key convenience feature.

Lessons for this Thesis: This overview of advanced robot features underscores the complexity and multi-layered sensing involved in achieving high levels of autonomy and

cleaning effectiveness. It clearly illustrates the progression from basic reactive behaviors to sophisticated mapping, navigation, and object recognition capabilities. For this project, which aims for a cost-effective prototype with core functionalities, this analysis provides a valuable benchmark and highlights potential pathways for future enhancements. While full SLAM with sensor fusion is beyond the current scope, understanding the role of individual sensors (like ultrasonic for basic detection, potential for IR cliff sensors, or even a simple camera in the future) and the importance of intelligent obstacle response logic is highly relevant. The discussion on efficient path coverage, even with simpler sensors, also informs the design of the autonomous navigation algorithm for this project.

2.2.2 Commercial Cleaning Robot Designs

Store-bought robots show what is possible in terms of performance, features, and ease of use. Looking at them helps us understand what people expect from these robots and see how advanced technologies are used.

Product 1: iRobot Roomba 980 [15, 16]

The iRobot Roomba 980 is a well-known high-quality robot vacuum cleaner, famous for its smart features and good cleaning.



Figure 2.9: Photo of iRobot Roomba 980 [15].

Key Features & Detailed Analysis:

- **Navigation (vSLAM):** The Roomba 980 uses an advanced navigation system with a camera. The camera, facing upwards, constantly takes pictures of the room, finding unique visual points (like patterns on the ceiling, edges of furniture, and walls). By tracking these points from picture to picture and combining this with its own movement information (often from wheel sensors and an IMU), the robot builds a map of the room and knows where it is on that map. This vSLAM ability allows it to clean in an organized and efficient way (like in straight lines, one room at a time) instead of just bouncing around randomly like simpler robots. It can usually clean a whole floor of a house and remember where it has already cleaned.

- **Adaptive Cleaning Technologies:**
  - **PowerBoost:** This feature automatically makes the vacuum stronger when the robot senses it's on a carpet. Sensors (maybe optical or by noticing if the motors are working harder) detect the carpet. This makes the vacuum more powerful to pick up dirt stuck in the carpet. On hard floors, it uses normal suction to save battery.
  - **Dirt Detect™ Series II Technology:** The Roomba 980 uses special sensors (optical and acoustic) to find areas with more dirt. When these sensors find a lot of dirt, the robot changes how it cleans that spot. It might go over the area several times or use a special pattern until it thinks the area is clean. This makes sure dirtier spots get cleaned better.
- **Other Important Features:**
  - **Edge-Cleaning Brush:** A spinning side brush sweeps dirt from along walls, under cabinets, and around furniture legs. It pushes the dirt into the path of the main brushes and vacuum.
  - **Wi-Fi and iRobot HOME App:** The robot connects to the home Wi-Fi. This lets users control and check on it from far away using a smartphone app. The app usually lets you start, stop, or pause cleaning; set up regular cleaning times; change cleaning settings (like how many times to go over an area); see maps of where the robot cleaned; and get messages.
  - **Automatic Recharging and Resuming:** If the battery gets low while cleaning, the Roomba 980 is designed to go back to its charging station by itself. After it has recharged enough, it can start cleaning again from where it stopped. This is helpful for cleaning large areas without needing someone to do anything.
- **Market Importance:** When the Roomba 980 came out, it really improved what people expected from robot vacuums. It showed how vSLAM could be used for good navigation in homes and introduced smart features like changing suction power and finding very dirty spots. These features helped iRobot become a leading company and changed what consumers thought a "smart" vacuum cleaner should do, especially regarding how well it cleans on its own, how smart it is, and how easy it is to use with an app.

Product 2: Xiaomi Robot Vacuum X20 Plus [17, 18]

The Xiaomi Robot Vacuum X20 Plus is an example of a robot that includes very new features, especially for navigation and taking care of itself. It often sells for a lower price than other top brands, making it a strong competitor. **Key Features & Detailed Analysis:**

- **Navigation (LDS / LiDAR):** The X20 Plus uses an LDS navigation system, which is a type of LiDAR. A laser turret on top of the robot spins quickly, sending out



Figure 2.10: Photo of Xiaomi Robot Vacuum X20 Plus [17].

laser beams and measuring the reflected light. This creates a very accurate 360-degree map of the room in real time. This technology allows for very precise distance measurements and map making. This helps the robot plan efficient and orderly cleaning paths, move around complex furniture, and work well even in dim light where cameras might have trouble. The maps it creates can often be seen in an app. Users can use these maps to set up virtual walls (to keep the robot out of certain areas), no-go zones, or choose specific spots for the robot to clean.

- **Hybrid Cleaning (Vacuuming and Mopping):** This model can do two jobs: it has strong vacuum suction and an attached mopping system. The vacuum system usually has high suction power (measured in Pascals) to pick up dust, hair, and bigger pieces of dirt. The mopping system might have a water tank and a microfiber mop pad. Advanced mopping features can include electronic control of water flow to change how damp the mop is, and sometimes special mopping movements (like a Y-shape scrubbing motion) to clean stains better.

- **Comprehensive Self-Maintenance Station:** A major feature of the X20 Plus (and similar high-end robots) is its all-in-one station that automatically empties the dustbin and washes the mop. This greatly reduces how much users have to do manually:

  - **Automatic Dustbin Emptying:** After cleaning, the robot goes to its dock. The station then uses strong suction to empty the robot's small dustbin into a much larger dust bag or container in the station. This means the robot can clean for weeks before the user needs to empty the station's container.
  - **Automatic Mop Washing:** The station can clean the robot's mop pads. This might involve parts like spinning scrubbers, fresh water sprays to wash the pads, and a system to collect the dirty water.
  - **Automatic Mop Drying:** After washing, the station often uses warm air to dry the mop pads. This stops mold and smells from growing and makes sure the mop is clean for the next use.

- **Market Importance:** The Xiaomi X20 Plus and similar robots show how quickly advanced features are becoming common and more affordable. The use of LiDAR for precise navigation and especially the smart self-maintenance stations solve big problems that users had with older robot vacuums (like having to empty the dustbin often and clean the mop by hand). This pushes the industry towards robots that are not just autonomous in cleaning, but also in their own upkeep, making them much more convenient and user-friendly.

These commercial examples demonstrate the sophisticated capabilities that are possible with advanced sensor suites, powerful processing, and refined algorithms. While this project aims for a more modest set of features due to cost and complexity constraints, understanding these high-end systems provides valuable context and inspiration for future development.

2.3 Key Takeaways and Problem Statement

Looking at both non-commercial projects and store-bought robots shows a wide and quickly changing world of cleaning robot technology. We can see a clear range:

- On one side, non-commercial projects show that it's possible to build basic cleaning robots with cheap and easy-to-find parts. These projects are often simple and can be changed to fit specific needs. However, they often don't move around very well, clean as effectively, or work as independently as more advanced robots. They usually just react to what's around them, and their cleaning tools can be very basic.

- On the other side, high-end store-bought robots are very advanced. They have many smart sensors (like LiDAR and cameras), complex software (for SLAM and AI-based path planning), powerful cleaning systems, and many automatic features, like charging themselves and emptying their own dustbins. But, these advanced features make them very expensive, so not everyone can afford them.

This review clearly shows a big gap in the market: there's a need for automated cleaning solutions that offer a good mix of features and affordability. Many people might want something better than basic custom-built kits but are not ready to pay for the most expensive commercial robots.

Problem Statement: This bachelor thesis project aims to fill this gap by designing and building a smart cleaning robot prototype that carefully balances useful features, ease of use, and reasonable cost. The main goal is to create a system using parts that are easy to find, affordable, and have good support for typical homes, especially those with common floor types found in places like Egypt. The core technical challenge this project addresses is achieving reliable and repeatable navigation on a budget. The main focus is therefore to implement **closed-loop motor control via wheel encoders**, moving beyond the unpredictable, timed-based movements common in simpler designs. This is complemented by BLE remote control for manual operation. The system is built around a Raspberry Pi Pico W microcontroller, HC-SR04 sensors for basic sensing, standard DC motors, and a repurposed fan for an effective vacuum. While fully independent mapping and path planning (like SLAM) are beyond the scope of this first prototype, this project

aims to create a robust foundation for more advanced features. It seeks a middle ground between simplistic DIY kits and expensive commercial robots, with a modular design that creates a clear path for future development in affordable home robotics.

Chapter 3

Mechanical Design and Assembly

3.1 Introduction

This chapter details the mechanical design and physical construction of the Smart Cleaning Robot prototype. It covers the overall structural design, the CAD-based development of individual components using SolidWorks, material considerations, and the assembly of the robot's chassis, the encoder-based drive system for precision tracking, and the integrated vacuum cleaning mechanism. The dimensions for the custom-designed parts in SolidWorks were determined based on the physical sizes of the selected electronic and mechanical components. A comprehensive Bill of Materials, including all components and associated costs, can be found in Appendix A. The design prioritizes functionality, ease of assembly for a prototype, and component housing while also considering the challenges encountered during fabrication and iteration. The overall dimensions of the assembled robot are approximately 35cm in diameter and 17cm in height (excluding the small wheel clearance). The fully assembled prototype has an approximate total weight of 2.65 kg.

3.2 Mechanical Components Summary

The key mechanical components providing the robot's structure and non-powered mobility are summarized in Table 3.1. This table focuses only on the structural parts; the full list of electrical components is detailed in Chapter 4, and a comprehensive Bill of Materials for all parts can be found in Appendix A.

Table 3.1: Mechanical Components List

Component	**Photo**	**Dimensions (cm)**	**Material**
Custom 3D-Printed Chassis		$\varnothing \approx 35$, H ≈ 17	PLA+

Table 3.1 – *Continued from previous page*

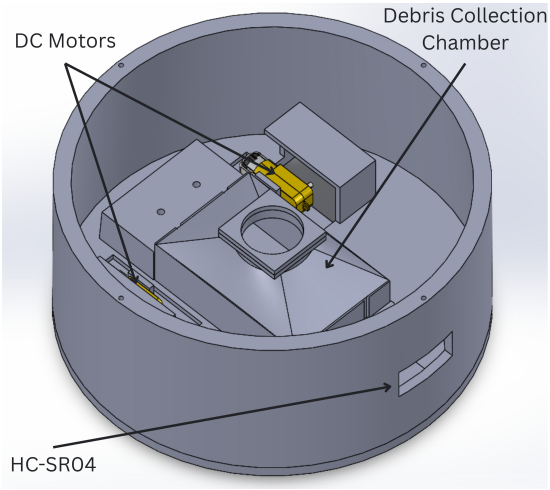
Component	Photo	Dimensions (cm)	Material
Custom 3D-Printed Debris Chamber		WxLxH: 10.5×16×8	PLA+
Robot Wheels		∅: 6.6, W: 2.8	Plastic Rim, Rubber Tyre
Metal Caster Wheel		Height: 2.0, Wheel ∅: 1.5	Steel Body and Ball
PCB Spacers (Snap-in Rivet)		Length: 2.0	Nylon
Bolts and Nuts (M3)		M3 (various lengths)	Steel

3.3 Detailed Mechanical Design

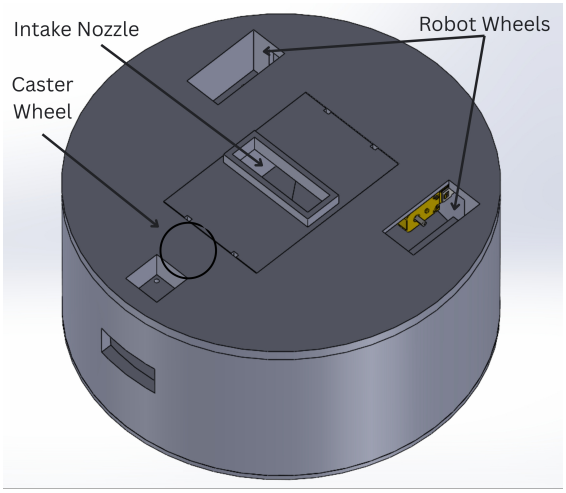
3.3.1 Chassis: Structure and Material

The main chassis forms the backbone of the robot, providing mounting points for all components and defining the robot's overall form factor. The robot features a generally circular chassis, a common design choice for cleaning robots as it aids in maneuverability and helps the robot glance off obstacles rather than getting caught on corners, facilitating navigation in typical household environments. The custom components were fabricated using PLA+ filament via an FDM 3D printing service; PLA+ was the material offered by the service and is suitable for prototypes due to its ease of printing and adequate strength. Its rigidity is beneficial for creating a stable chassis that prevents flex, ensuring consistent alignment for the drive system and sensors. A substantial wall thickness of approximately 1 cm was utilized for the main structural components to ensure robustness and resistance to impacts during operation. (Refer to Figure 3.1 for detailed CAD views).

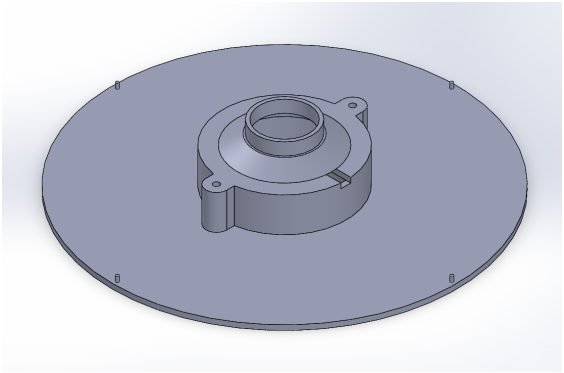
The chassis is a custom design modeled in SolidWorks and 3D printed. It features:



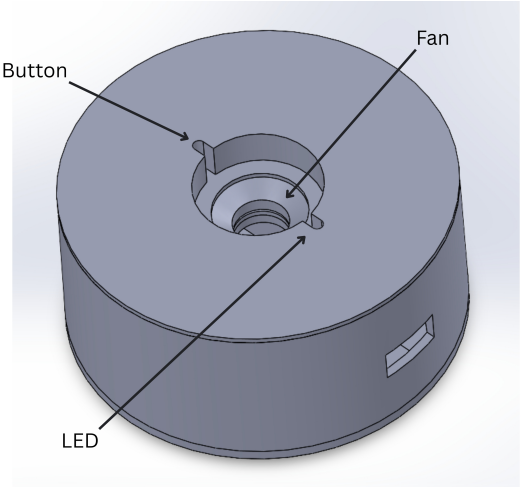
(a) Isometric view of the open chassis.



(b) Bottom CAD view of the chassis.



(c) Isometric view of the main robot lid.



(d) Fully assembled robot isometric view.

Figure 3.1: SolidWorks CAD models illustrating the Smart Cleaning Robot's chassis and assembled form, with annotations to highlight key features.

- **Base Plate**: Provides the primary mounting surface for motors, the caster wheel, and the Debris Collection Chamber. (See Figure 3.1b and 3.1a). The 12V Li-ion battery pack is secured between one of the debris chamber's guide blocks (discussed later) and a chassis wall, positioned towards the rear above the drive wheels to enhance traction.

- **Wheel Housings and Motor Mounting Points**: Located at the rear of the chassis, dedicated structures provide partial housing for the wheels. The two DC gear motors are screwed into small, integrated walls on the chassis; these walls incorporate precise mounting holes and provide lateral support and alignment for the motors. (Visible in Figure 3.1a). These structures were also designed to provide stable mounting points for the IR photoelectric sensors, ensuring the encoder discs that rotate with the wheels pass correctly through their sensing grooves.

- **Sensor Cutout and Mounting**: A dedicated cutout at the front of the chassis houses the HC-SR04 sensor. A custom bracket was designed to securely hold the sensor within this cutout, ensuring correct alignment for obstacle detection. (Visible in Figure 3.1a and 3.1d).

- **Front Caster Wheel Mount**: The front bottom section of the chassis includes a mounting point for the caster wheel. Due to the unavailability of the initially planned component, an alternative caster wheel was utilized, necessitating an adapted screw mounting method on the chassis base. (See Figure 3.1b and Figure 3.2).

- **Central Cutout**: A large central cutout accommodates the Debris Collection Chamber. (Visible in Figure 3.1b).

- **Electronics Mounting**: The custom PCB (which hosts the Raspberry Pi Pico W, LM2596S buck converter, and sensor voltage divider resistors) and the L298N H-bridge motor driver module are elevated from the chassis base using 20mm nylon snap-in rivet PCB spacers. This provides clearance for wiring and protection for the electronics.

3.3.2 Vacuum System Design

The Debris Collection Chamber is a key sub-assembly designed to collect vacuumed particles. The chamber was designed for a tight clearance fit within the chassis bay to ensure stability and prevent rattling. (See Figure 3.3 for its CAD design and Figure 3.4 for a photo of the 3D printed part).

- **Chamber Design**: The chamber itself is a custom 3D-printed component. It features a rectangular box base measuring 10.5 cm in width, 16 cm in length, and 5.5 cm in height. From this base, the chamber transitions through a pyramid-like section for an additional 2.5 cm in height, culminating in a circular outlet of 5 cm in diameter. This outlet is designed to interface with the vacuum fan and also seats the air filter. The debris inlet, located at the bottom of the chamber, has dimensions of 8.5 cm x 2.5 cm.

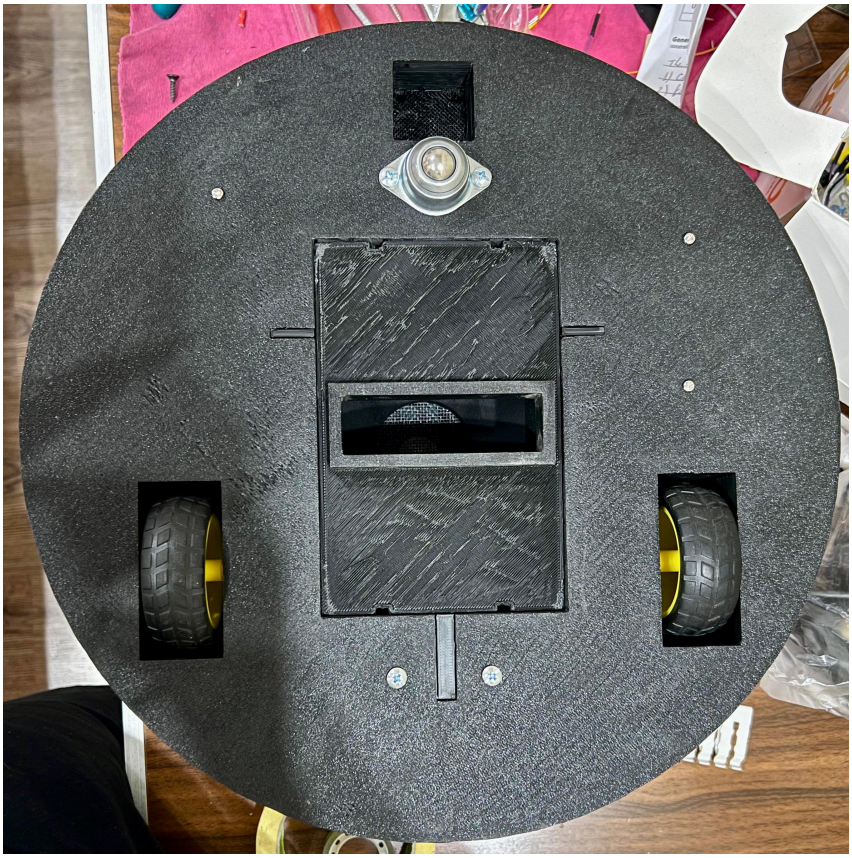


Figure 3.2: Bottom view photograph of the assembled Smart Cleaning Robot, showing the caster wheel, drive wheels, and vacuum intake nozzle.

- **Mounting and Guides**: The method for positioning the chamber within the chassis was refined during development (detailed in Section 3.4). The final design uses two robust 3D-printed blocks screwed to the chassis base, incorporating guide elements for secure and precise placement.
- **Filter and Lid**: A removable lid is designed for the top of the Debris Collection Chamber to hold an air filter. This lid fits into corresponding features on the chamber for easy removal and a secure fit. The main robot chassis lid (Figure 3.1c), when closed, presses down on this filter lid, keeping it secure.

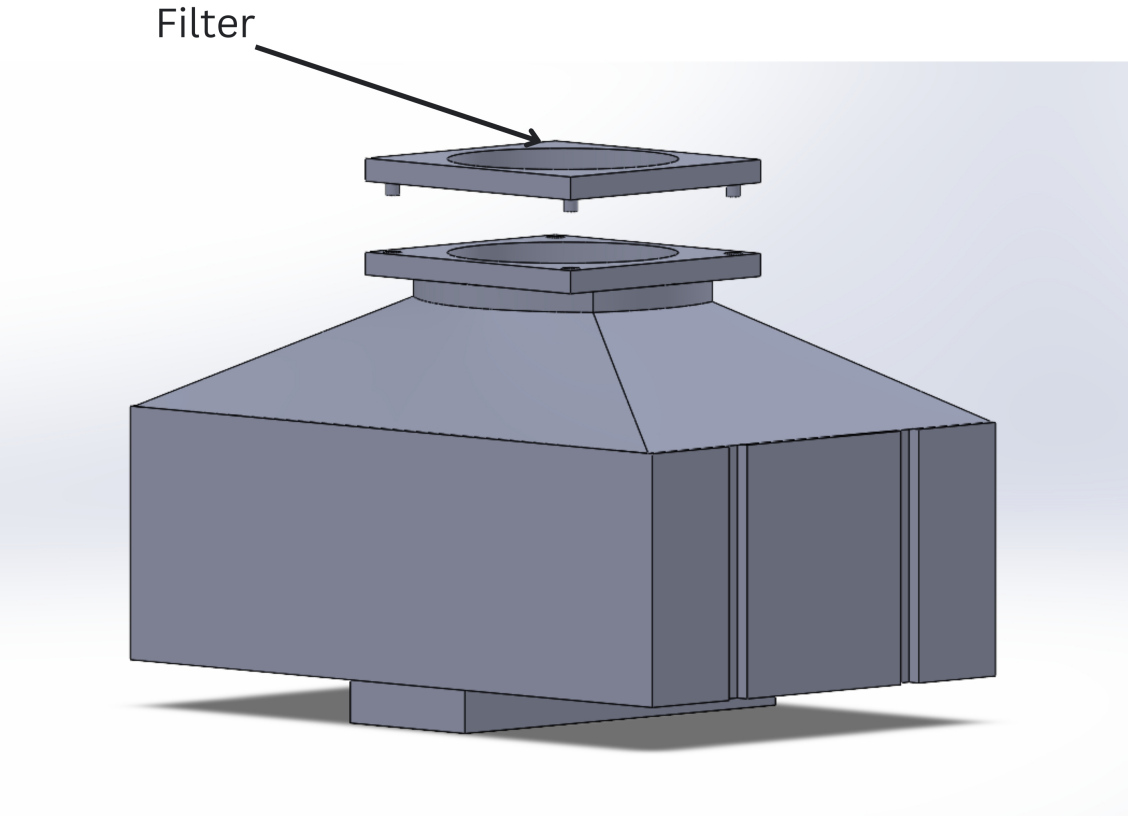


Figure 3.3: SolidWorks CAD design of the Debris Collection Chamber.

The vacuum system is responsible for generating airflow to collect debris.

- **Fan Selection and Mounting**: The primary vacuum source is a repurposed PS3 fan (Model: Delta KSB1012HE) [23]. This fan was readily available and met basic size and 12V DC voltage requirements. It has a diameter of approximately 10cm and features 17 blades. The fan is mounted within the main top lid of the robot (see Figure 3.1c).



Figure 3.4: Photograph of the 3D printed Debris Collection Chamber with its removable filter lid.

- **Airflow Path**: Air is drawn through an intake nozzle (part of the Debris Collection Chamber design, Figure 3.3), passes through the filter in the chamber's lid, then through the fan, and is exhausted from the robot. The debris chamber was designed to channel airflow with minimal obstruction.

3.3.3 Drive System Design

The drive system enables the robot's mobility. A differential drive configuration was selected for its simplicity in control, maneuverability (allowing for zero-radius turns), and common availability of components.

- **Motors and Wheels**: Two DC gear motors (TT type) provide locomotion, each driving a 66mm diameter wheel. These are mounted in dedicated compartments at the rear of the chassis. (Visible in Figure 3.1a and 3.1b).
- **Encoder Integration**: To enable precise movement control, a 20-slot plastic encoder disc was press-fit onto the interior side of each wheel's axle. An IR photoelectric sensor was mounted on the chassis such that the encoder disc passes through its 5mm groove. This setup allows for the measurement of wheel rotation, as detailed in Figure 3.5.
- **Caster Wheel**: A caster wheel at the front provides a third point of contact, allowing for maneuverability and stable turns. As noted, the mounting of this wheel was adapted due to component availability. (Visible in Figure 3.1b and Figure 3.2).

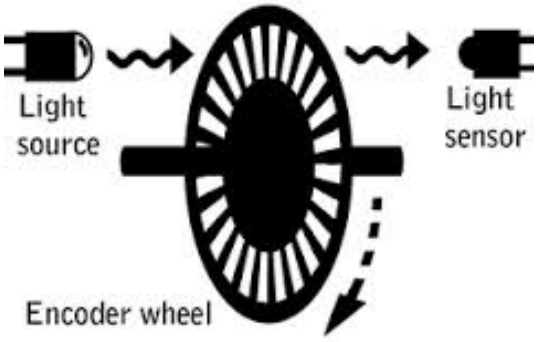


Figure 3.5: Diagram illustrating the principle of operation for the optical wheel encoder. As the encoder wheel (slotted disc) rotates with the main robot wheel, its slots alternately block and pass the light beam from the source to the sensor, generating a digital pulse train that corresponds to the distance traveled.

3.3.4 Main Robot Lid and Component Fastening

The top lid of the robot (Figure 3.1c) serves multiple functions: it encloses the internal components, supports the vacuum fan assembly, presses down to secure the filter lid of the Debris Collection Chamber, and provides mounting points for an LED indicator and a power button. The main top lid incorporates dedicated openings for the power button and LED indicator, positioned conveniently next to the fan exhaust; these components are secured via friction fit. The method for securing the main top lid to the chassis was refined from an initial concept of small 3D-printed latching pieces to a more robust solution using four screws (detailed in Section 3.4). The accessibility of the battery for charging was considered in the overall assembly.

3.4 Mechanical Assembly Implementation

The implementation process for the mechanical structure involved the fabrication of custom parts using an FDM 3D printer with PLA+ filament (provided by a 3D printing service), and the assembly of all mechanical components onto the chassis.

The mechanical assembly followed a structured sequence:

1. The two DC gear motors and the front caster wheel were first secured to the 3D-printed chassis base.

2. The custom 3D-printed Debris Collection Chamber was then slotted into its designated bay within the chassis.

3. Electronic components, including the custom PCB (hosting the Raspberry Pi Pico W, LM2596S buck converter, and associated circuitry) and the L298N motor driver, were mounted onto their respective PCB spacers, which were affixed to the chassis. (The internal arrangement and wiring are detailed in Chapter 4, Figure 4.4).

4. Essential wiring connections between the motors, sensors, PCB, and power components were then routed and made.

5. The PS3 fan was installed into its housing within the main top lid.

6. Finally, the top lid, with the fan, power button, and LED indicator secured, was fastened to the main chassis using four M3 screws, enclosing the internal components.

An iterative design approach was crucial for refining certain mechanical aspects. For instance, the initial design for positioning the Debris Collection Chamber using slender guides proved too fragile. This was rectified by redesigning the mounting system to use two larger, screwed-in blocks with improved guide elements for precise placement and durability. Similarly, the main top lid fastening was changed from small latching pieces, found to be insufficiently robust against fan vibrations, to a more secure four-screw system. Minor adjustments, such as reaming mounting holes, were made as needed during assembly to ensure proper alignment and securement, a common practice in prototyping.

Figure 3.6 shows photographs of the fully 3D printed and assembled robot prototype. (The internal view showing component wiring and PCB integration is presented in Chapter 4, Figure 4.4).



(a) Isometric view of the assembled robot. (b) Top view of the assembled robot.

Figure 3.6: Photographs of the 3D printed and assembled robot prototype.

Chapter 4

Electrical and Control System Design and Testing

4.1 Introduction

This chapter details the electronic and software aspects of the Smart Cleaning Robot. It begins by outlining the key electrical components used and then describes the overall system architecture. The chapter then delves into the electrical design, including power distribution, component interconnections, and the custom PCB that forms the heart of the robot's electronics. Following this, the control system implementation is discussed, covering the MicroPython software developed for the Raspberry Pi Pico W, its architecture, navigation logic, and BLE remote control capabilities. Finally, the chapter presents a qualitative performance evaluation of the fully assembled and programmed prototype.

4.2 Electrical Components Summary

The core electrical and electronic components enabling the robot's functionality are listed in Table 4.1. These components are responsible for power management, sensing, control, and actuation. A comprehensive Bill of Materials, including all components and associated costs, is available in Appendix A.

Table 4.1: Electrical Components List

Component	**Photo**	**Dimensions (cm)**	**Electrical Specifications**
Raspberry Pi Pico W [9, 19]		PCB: 2.1×5.1	RP2040, Dual-core ARM Cortex-M0+ @133MHz **Voltage:** 1.8–5.5V, 3.3V logic **Current:** 91 mA (Wi-Fi active) **Power:** Max 0.5W

Table 4.1 – *Continued from previous page*

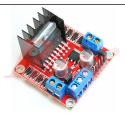
Component	Photo	Dimensions (cm)	Electrical Specifications
DC Motors (TT Mini Gearbox Pair)		Per motor: W2.2 ×L7×H1.8	**No-load Speed:** 90 RPM **Max Torque:** 800 gf.cm **Voltage:** 3-12V **No-load Current:** 190-250mA
H-Bridge L298N Motor Driver [20, 21]		4.3×4.3×2.7	**Logic Voltage:** 5V **Drive Voltage:** 5-35V **Max Current:** 2A/channel **Max Power:** 25W
HC-SR04 Sensor [7]		W4.45 ×H2×D1.5	**Voltage:** 5V DC **Current:** ∼15mA **Range:** 2cm - 400cm
Battery Pack [22]		9.3×6×1.8	**Output:** DC 2.1mm Jack **Type:** Lithium-ion **Capacity:** 4000mAh @ 12V
PS3 Fan [23]		Diameter: ∼10	**Voltage:** 12V DC **Current:** ∼1.30A **Connection:** 3-Pin (Pwr, Gnd, PWM Ctrl)
DC-DC Step Down LM2596S [24]		4.35×2.1×1.4	**Input:** 4-38V DC **Output:** 1.5-35V DC (Adj.) **Current:** 3A Max **Frequency:** 150kHz

Table 4.1 – *Continued from previous page*

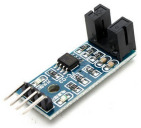
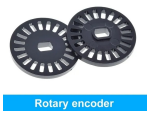
Component	Photo	Dimensions (cm)	Electrical Specifications
IR Photoelectric Encoder Sensor		3.2×1.4×1.0	**IC:** LM393 **Voltage:** 3.3-5.0V **Groove Width:** 5mm **Output:** Digital TTL
Rotary Encoder Disc		∅: 2.5	**Material:** Plastic **Slots:** 20
Push Button (Green)		Thread ∅: 9mm Button ∅: 7mm Overall: 1.2×2.5	**Type:** Momentary, Normally Open **Rating:** AC 3A/125V, 1.5A/250V
LED Indicator		Mount Hole ∅: 10mm Overall: L4.15×∅1.15	**Rated Voltage:** DC 12V **Color:** Green
2 Pin Screw Terminal		Pitch: 0.5 Pin Dim: 0.43	**Rating:** 24A / 250V **Wire:** 26-14AWG
3 Pin Screw Terminal		Pitch: 0.5 Pin Dim: 0.43	**Rating:** 24A / 250V **Wire:** 26-14AWG
Pin Header Female 1x20		Spacing: 0.254	**Configuration:** 1x20 Pins, Straight **Current Rating:** 3A

Table 4.1 – *Continued from previous page*

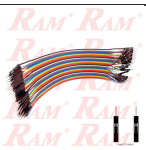
Component	Photo	Dimensions (cm)	Electrical Specifications
Heat Shrink Sleeve (6mm Yellow)		∅: 0.6 Length: 4.0	**Material:** Polyolefin **Shrink Ratio:** 2:1, Waterproof
Resistor 2.2kΩ		Axial Pkg.	**Value:** 2.2kΩ **Power:** 1/4W **Type:** Axial
Resistor 4.7kΩ		Axial Pkg.	**Value:** 4.7kΩ **Power:** 1/4W **Type:** Axial
Jumper Wires M-F		Length: 20	**Configuration:** Male-to-Female **Quantity:** 40 Wires **Current Rating:** 1A
Jumper Wires M-M		Length: 20	**Configuration:** Male-to-Male **Quantity:** 40 Wires **Current Rating:** 1A
Jumper Wires F-F		Length: 20	**Configuration:** Female-to-Female **Quantity:** 40 Wires **Current Rating:** 1A
Speaker Wire		Size: 2Cx0.1 Length: 100 (per meter)	**Colors:** Red & Black **Material:** OFC Copper, PVC Jacket

Table 4.1 – *Continued from previous page*

Component	Photo	Dimensions (cm)	Electrical Specifications
		System Voltage:	12V
		Approx. Operational Current:	2.2A
		Approx. Operational Power:	26W

4.3 System Architecture

The robot's control system is centralized around the Raspberry Pi Pico W microcontroller [9]. This board was chosen for its dual-core ARM Cortex-M0+ processor, sufficient GPIO pins, integrated Wi-Fi and BLE capabilities (Pico W variant), low cost, and robust MicroPython support [25]. A pinout diagram for the Raspberry Pi Pico W is shown in Figure 4.2. The Pico W is responsible for:

- Reading and processing data from the HC-SR04 ultrasonic sensor for obstacle detection.
- Processing high-frequency interrupt signals from two wheel encoders for odometry.
- Controlling the two DC gear motors via an L298N H-bridge motor driver, using a PID algorithm for precise motion control.
- Managing the operation of the 12V PS3 fan.
- Handling user input from a physical start/stop button.
- Providing status feedback via an LED indicator.
- Managing BLE communication for remote control via a smartphone application like NRF Toolbox.

Figure 4.1 provides a high-level overview of the autonomous cleaning control flow, illustrating how the system transitions between user commands, forward motion with hybrid feed-forward and PID control, obstacle detection, and stall recovery.

4.4 Electrical Design and Integration

The electrical design focuses on safely and reliably interconnecting all electronic components, culminating in a custom PCB for a robust final assembly.

4.4.1 Power Distribution

- The 12V Li-ion battery [22] is the primary power source.

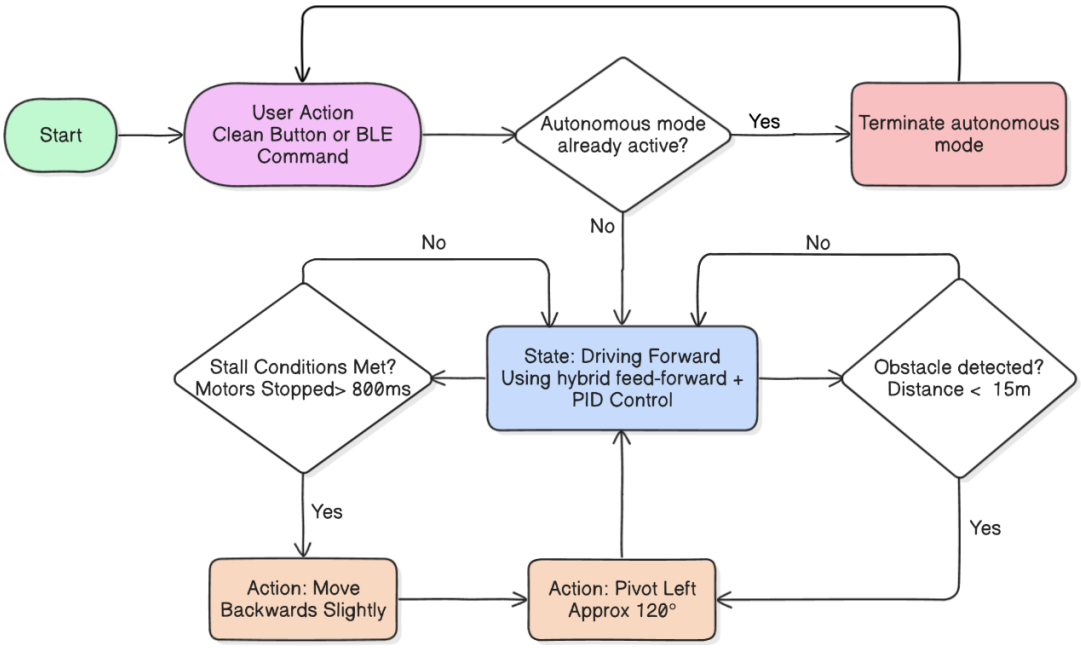


Figure 4.1: High-level control flowchart of the autonomous cleaning mode, showing user-initiated activation, forward motion with hybrid PID control, obstacle avoidance, and stall recovery paths.

- The LM2596S buck converter [24] steps down the 12V to a regulated 5V. This 5V rail powers:
  - The Raspberry Pi Pico W via its VSYS pin.
  - The logic side of the L298N motor driver (Vss or +5V pin).
  - The HC-SR04 ultrasonic sensor (VCC pin).
  - The two IR photoelectric encoder sensors (VCC pins).
- The 12V from the battery directly powers:
  - The motor input (Vs or VM) of the L298N module.
  - The PS3 fan.
- A common ground (GND) is maintained across all components.

4.4.2 Component Interconnections and Wiring

The Raspberry Pi Pico W's GPIO pins are used for all control signals and sensor inputs. Key connections are detailed in the accompanying MicroPython source code (`main.py`) and visualized in the schematic diagrams (see Section 4.4.3). The specific GPIO assignments are as follows:

- **Motor Control (L298N)**: Six GPIO pins from the Pico W control the L298N driver [20]:

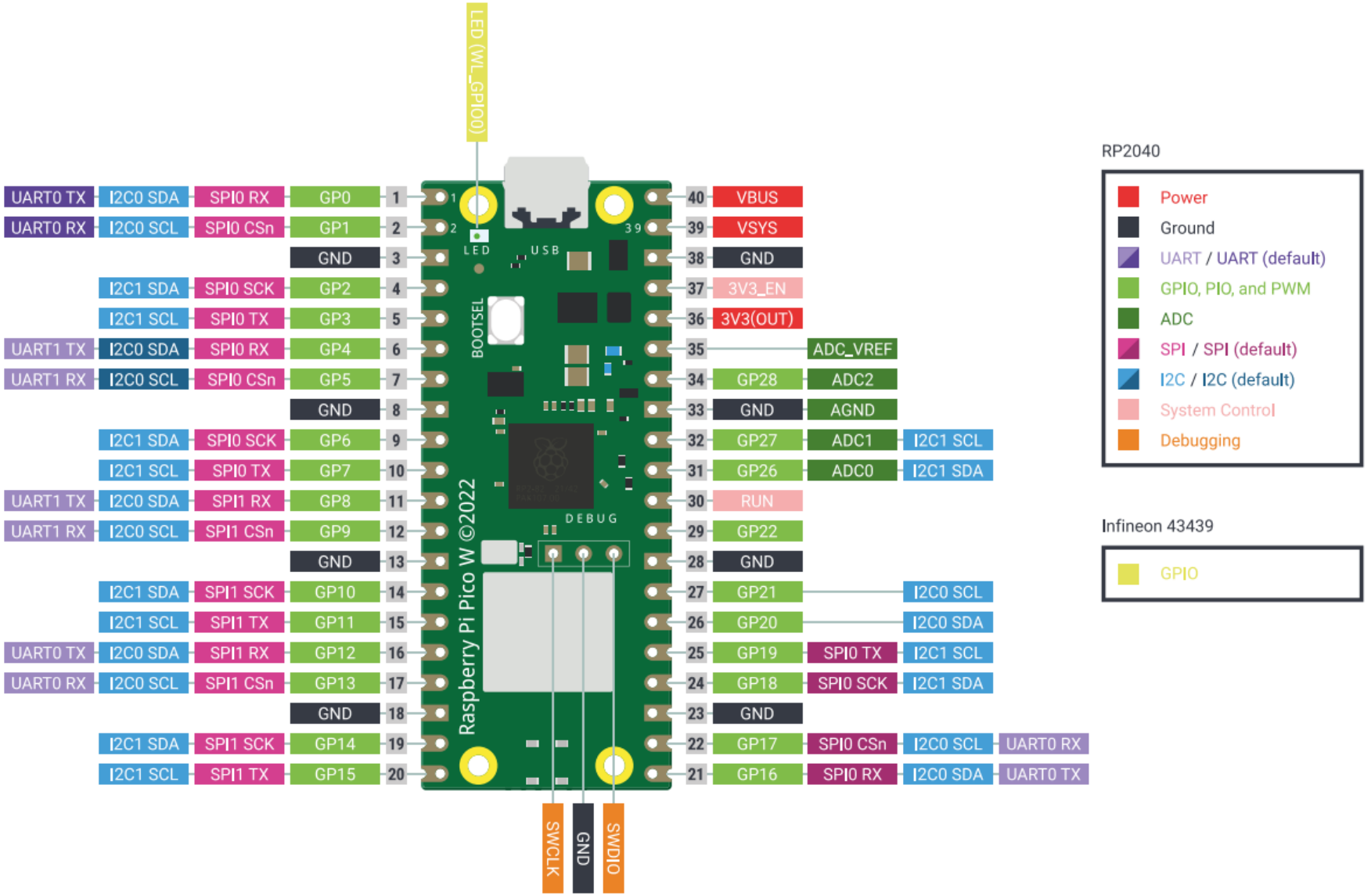


Figure 4.2: Raspberry Pi Pico W pinout diagram. Source: [9].

  - Left Motor PWM (ENA): GP6
  - Left Motor Direction 1 (IN1): GP12
  - Left Motor Direction 2 (IN2): GP13
  - Right Motor PWM (ENB): GP7
  - Right Motor Direction 1 (IN3): GP14
  - Right Motor Direction 2 (IN4): GP15

- **Wheel Encoder Inputs**: To facilitate the closed-loop control system, the wheel encoders were connected directly to interrupt-capable GPIO pins:
  - Left Encoder Sensor: GP21
  - Right Encoder Sensor: GP20

  These pins were configured with internal pull-ups in the software to ensure a stable signal.

- **Ultrasonic Sensor (HC-SR04)**:
  - TRIG (Output): GP10
  - ECHO (Input): GP11

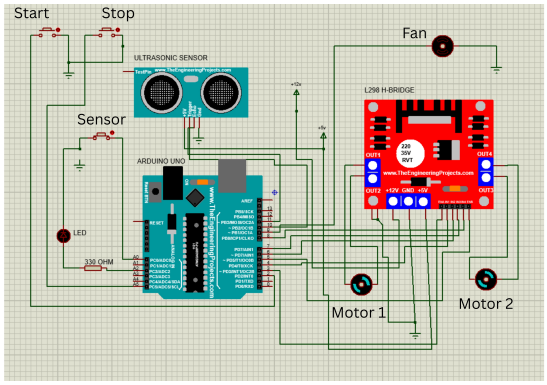
  A voltage divider circuit (e.g., 2.2kΩ and 4.7kΩ resistors) is used on the ECHO pin to safely step down the sensor's 5V output to the Pico W's 3.3V logic level.

- **PS3 Fan**: The fan's PWM control pin (Grey wire) is connected to GP8 on the Pico W. While this hardware connection could support future software-based PWM speed control, the current prototype implements simple on/off control using this pin (leaving software-based speed control as a potential future enhancement). The fan's +12V (Brown) and GND (Black) wires are connected to the 12V power supply.

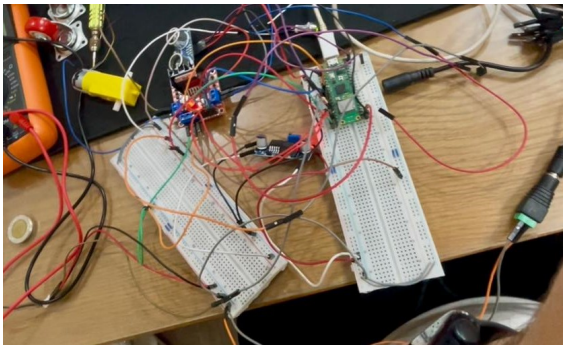
- **User Interface**:
  - Push Button (Start/Stop): GP16 (configured with an internal pull-up resistor).
  - LED Indicator: GP18 (connected via a suitable current-limiting resistor as required).

Initial conceptual validation of the control logic, particularly for obstacle avoidance, was performed using Proteus simulation (Figure 4.3(a)). An Arduino Uno model was employed in the simulation due to the unavailability of a Raspberry Pi Pico W model in Proteus at the time of testing; this allowed for early verification of the fundamental sensor-driven decision-making processes. Following this simulation-based logic testing, the actual electrical connections for the chosen components were physically validated on a breadboard (Figure 4.3(b)) before committing to the PCB design.

With the mechanical structure assembled, final electrical connections were made. Appropriate gauge wires were used for signal and power lines. Connections were double-checked for polarity and correctness against the wiring diagrams. Wire management involved routing cables to prevent interference with moving parts. Figure 4.4 shows the internal wiring with the custom PCB.



(a) Proteus simulation of basic obstacle avoidance logic.



(b) Breadboard prototype for physical validation.

Figure 4.3: Conceptual validation stages: (a) Proteus simulation of the robot's basic obstacle avoidance logic using an Arduino model, and (b) photograph of the breadboard prototype showing key component interconnections for physical validation.

4.4.3 Custom PCB Design and Fabrication

A custom 2-layer PCB (100mm x 50mm, FR4, 1oz copper, green solder mask, black legend) was designed in KiCad to integrate the Raspberry Pi Pico W, the LM2596S buck converter, screw terminals for external connections (motors, fan, sensor, button, LED), and the HC-SR04 voltage divider circuit. This provided a more robust and organized solution than breadboard or perfboard wiring for the final prototype. The custom PCB served as the central point for connections, utilizing screw terminals for robust wiring to off-board components. The schematic, layout, and a 3D render of the PCB are shown in Figures 4.5, 4.6, and 4.7 respectively.

It is important to note that the wheel encoders were integrated into the design as a post-manufacturing upgrade to enhance the robot's capabilities. Consequently, their wiring was connected directly to the Raspberry Pi Pico W's GPIO pins and is not reflected in the PCB schematic or layout diagrams (Figures 4.5, 4.6).

4.5 Control System Implementation (Software)

The robot's behavior is governed by software developed in MicroPython, running on the Raspberry Pi Pico W. The complete source code is provided as an accompanying file (`code_backup/main.py`).

4.5.1 Software Architecture

The software is structured modularly to handle various tasks:

- **Hardware Abstraction Layer**: Functions to initialize and interact with hardware peripherals, including hardware interrupt handlers for the wheel encoders.
- **PID Control Module**: Contains the feed-forward PWM calculation (`ff_pwm`), straight-line motion with positional PID (`straight_motion`), and pivot turns

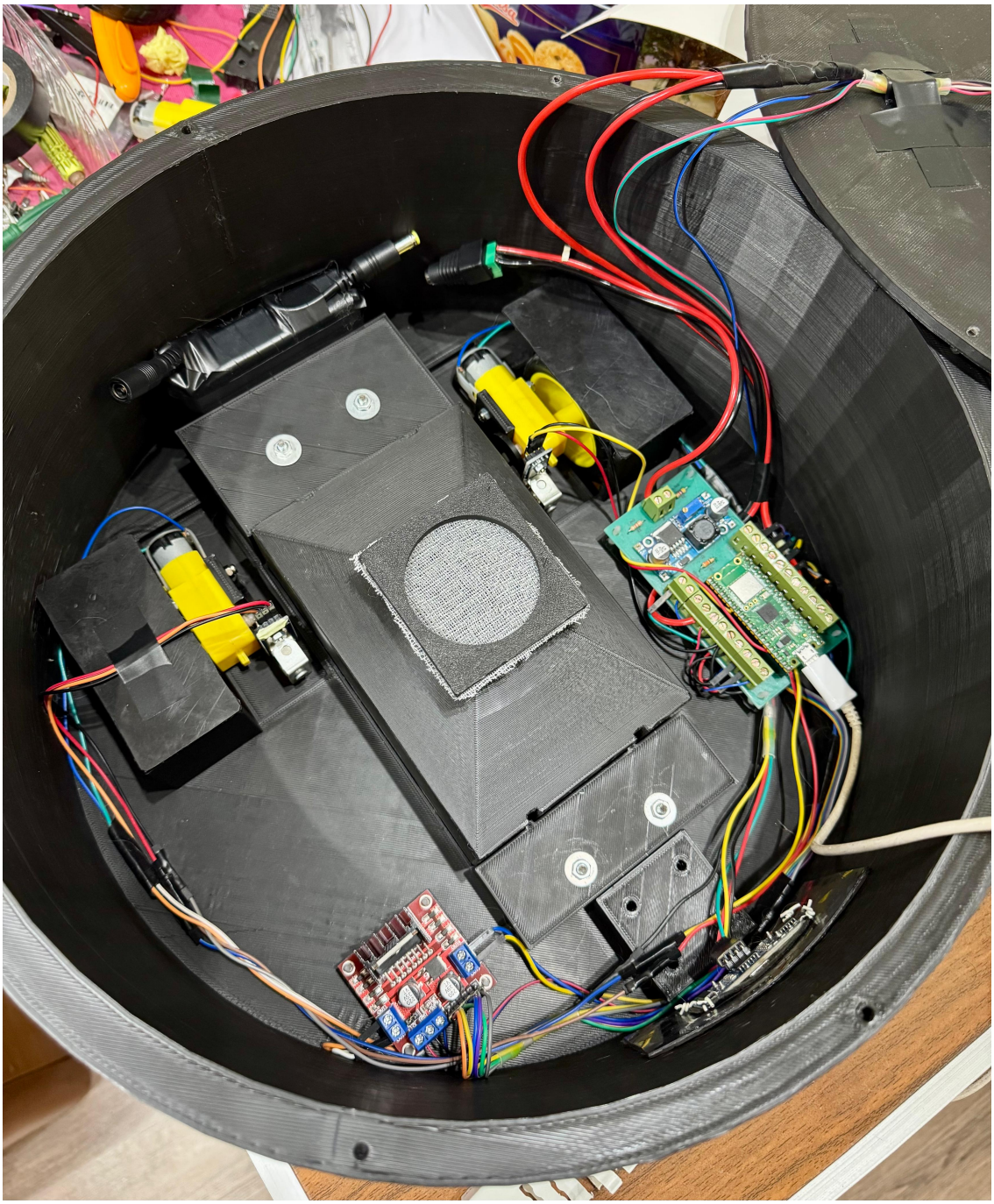


Figure 4.4: Internal view of the assembled robot showing component wiring and integration with the custom PCB.

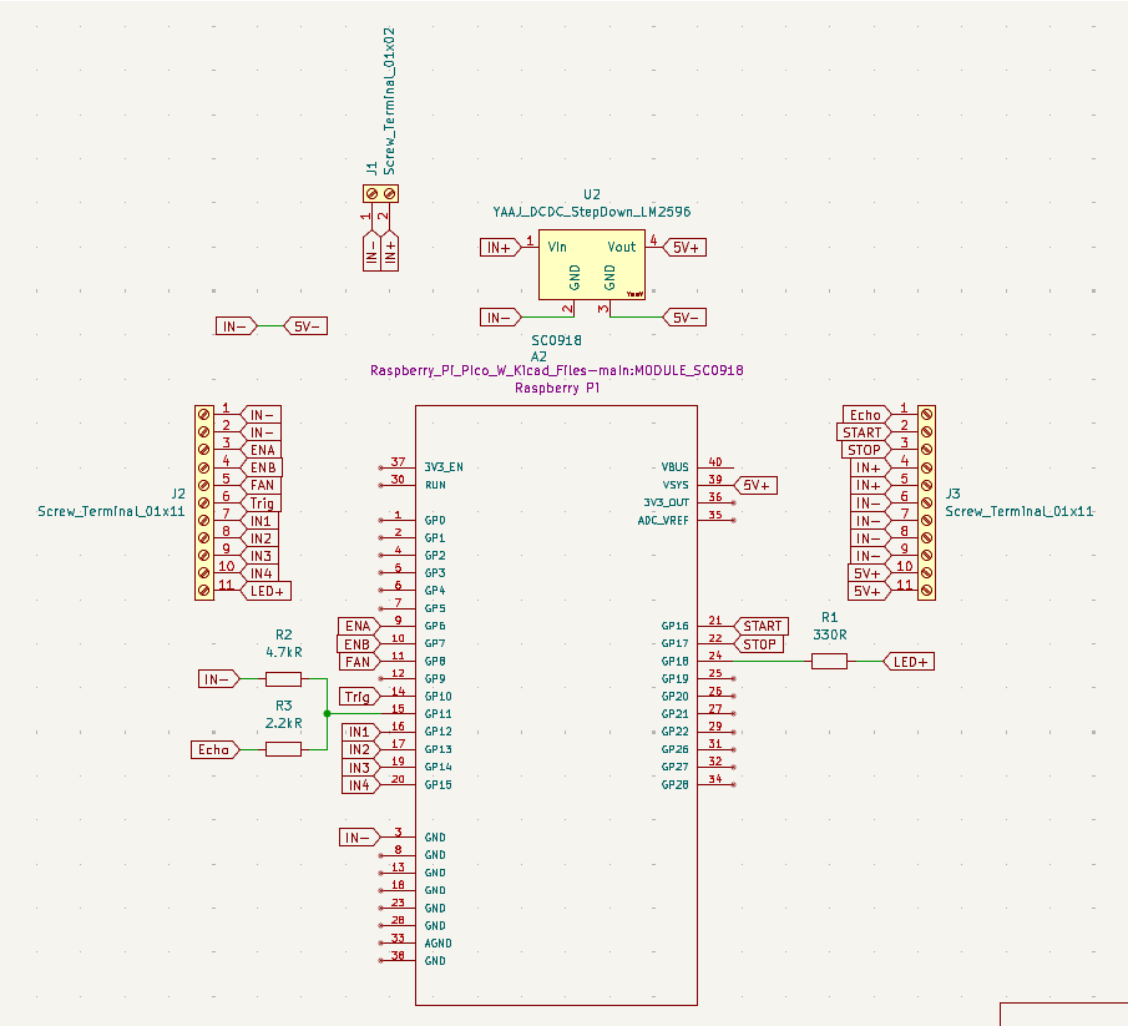


Figure 4.5: KiCad schematic diagram for the custom Printed Circuit Board.

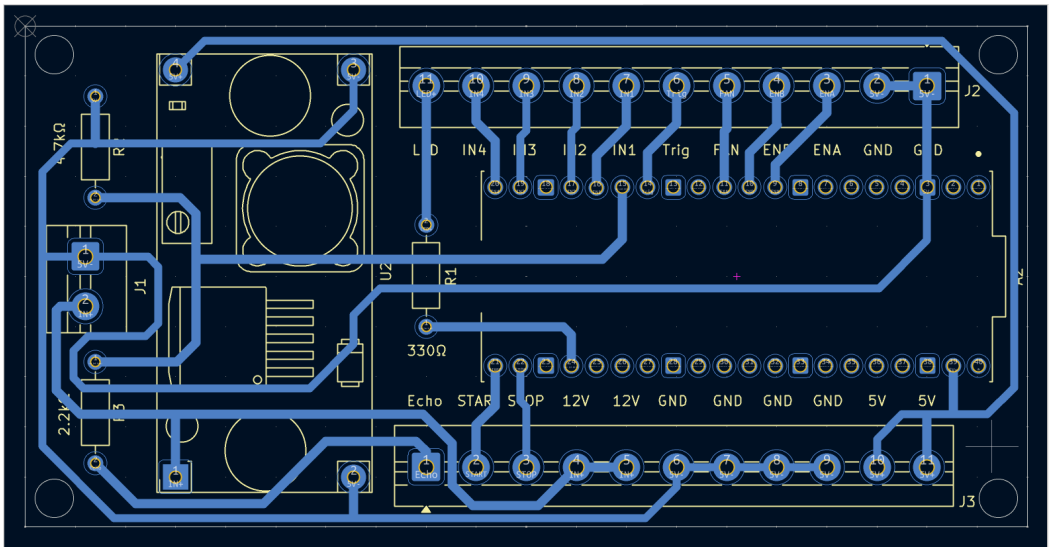


Figure 4.6: KiCad PCB layout design (2D view).

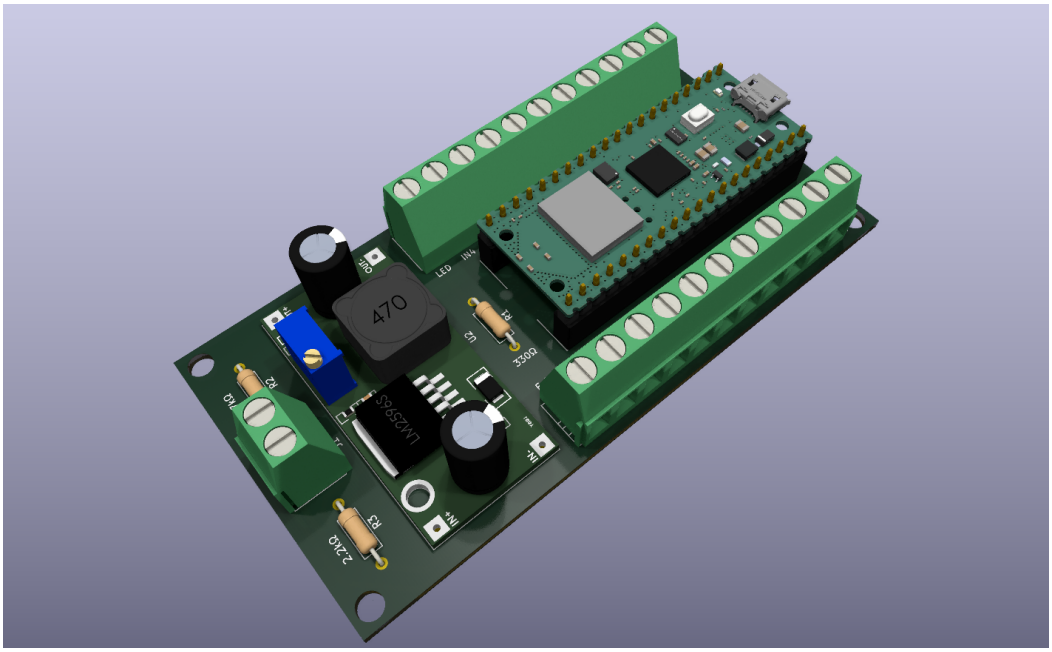


Figure 4.7: 3D rendering of the populated PCB design from KiCad.

with a dual progress/balance PID (`pivot_motion`).

- **Autonomous State Machine**: Implements the primary logic for autonomous cleaning as a finite state machine: `IDLE` → `START_FWD` → `FWD` (moving forward until an obstacle is detected) → `START_AVOID` → `AVOID` (executing a wide turn), with a `STALL_TURN` recovery state triggered by motor stall detection.
- **Safety and Monitoring Layer**: Includes critical functions for motor stall detection, active braking, and an emergency stop protocol to ensure safe operation.
- **User Interface Module**: Manages inputs from the physical start/stop button and the BLE remote, and controls the status LED.
- **Main Control Loop**: Uses a priority-based execution model—first handling horn/obstacle blocking, then manual commands with toggle behavior, then the autonomous finite state machine—while continuously updating the LED feedback system.

4.5.2 Evolution of the Control Strategy

The initial navigation strategy was purely reactive and time-based, which lacked precision. The introduction of wheel encoders enabled a more sophisticated, closed-loop system. The development evolved from a pure feedback approach to a more robust hybrid model.

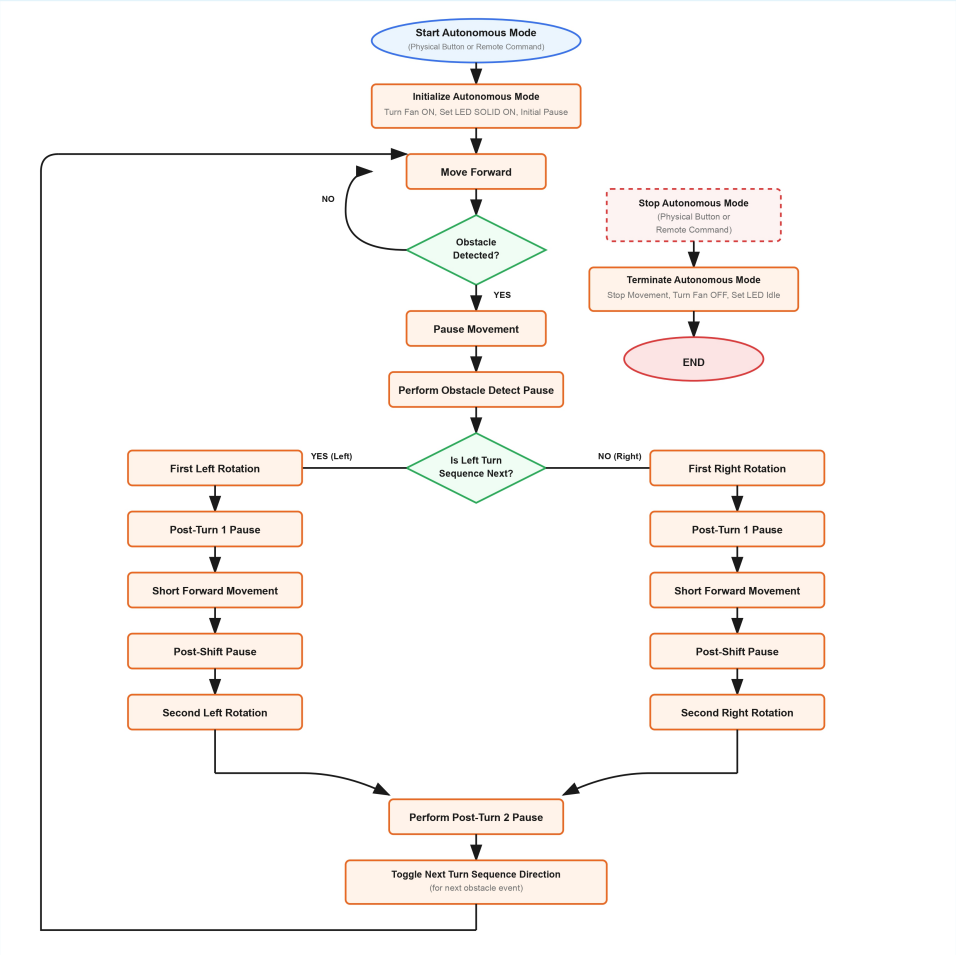


Figure 4.8: Block diagram of the autonomous obstacle-avoidance state machine showing the robot's operational states and transitions.

Limitation of Pure PID Control

The first closed-loop attempts used a **Positional PID** controller. This controller's error term was calculated from the difference in the total distance traveled by each wheel (`left_pulses - right_pulses`) to actively correct for drift. While a significant improvement, this purely reactive system was susceptible to a critical hardware limitation: aggressive PID adjustments, especially during turns, would attempt to draw excessive current to compensate for motor differences. This led to voltage sags (**brownouts**) that would stall a motor, causing the control system to fail catastrophically. This discovery highlighted that a superior software algorithm can be defeated by the physical limitations of its hardware.

Final Implemented Architecture: Hybrid Feed-Forward and PID Control

To create a more robust system, a **hybrid feed-forward and feedback** architecture was developed, as reflected in the accompanying source code (`main.py`).

- **Feed-Forward (FF) Model**: A simple linear model (`ff_pwm`) was created to proactively estimate the required PWM signal for a given target speed (in pulses per second). This forms the primary, anticipatory part of the controller, providing the "muscle" for a given movement based on the desired outcome.
- **PID as a Correction Layer**: The positional PID controller was retained but repurposed. Instead of calculating the entire motor command, it now calculates a small *adjustment* to the target speed. This feedback layer acts as a fine-tuner, correcting for small, unmodeled errors (e.g., wheel slip, friction changes) without making the large, aggressive power demands that previously caused system instability.
- **Manual Control Enhancement**: The manual control logic was updated to a toggle system. Sending a movement command (e.g., "FORWARD") initiates the motion; sending the same command a second time applies the brake and stops the robot, providing more intuitive control.
- **Stall Detection and Recovery**: A safety-critical routine was integrated. During autonomous movement, the software checks if the encoder pulses have stopped. If the robot is physically stuck, it triggers a recovery state where it reverses and turns to free itself.

This journey from a simple reactive system to a hardware-aware hybrid controller demonstrates a comprehensive engineering process of design, testing, diagnosis, and refinement.

PID Tuning Methodology

The PID gains (K_p, K_i, K_d) were determined through an empirical trial-and-error process, a practical approach suitable for this prototype. The tuning followed a systematic procedure:

1. **Start with P only**: Initially, only proportional gain was applied. The value was increased until oscillation occurred, then reduced.

2. **Add D for damping**: Derivative gain was introduced to reduce overshoot and smooth the response.

3. **Add minimal I**: A small integral term was added to eliminate steady-state drift, with careful limiting to prevent windup.

The final tuned values for the positional correction PID were conservative (low gains) to avoid the brownout conditions described earlier. This trade-off prioritized system stability over aggressive correction, accepting slightly less precise correction in exchange for reliable operation.

4.5.3 BLE Remote Control

The Raspberry Pi Pico W's BLE capability is utilized for manual remote control. A BLE UART service is implemented on the Pico W, allowing it to receive commands from a connected BLE central device, typically a smartphone running a compatible application.

NRF Toolbox Application

For this project, the NRF Toolbox application (or a similar BLE UART terminal app) can be used to interact with the robot. The application connects to the Pico W's BLE service and allows the user to send text-based commands over the UART profile.

- **Connection**: The user scans for BLE devices and connects to the robot, which advertises with a unique name (e.g., "PicoRobot-3C33").

- **Command Interface**: The UART interface in the NRF Toolbox app provides a console or programmable buttons to send specific string commands to the robot. The MicroPython code on the Pico W (`main.py`) is programmed to recognize commands such as "FORWARD", "BACKWARD", "LEFT", "RIGHT", "CLEAN", "FAN", and "HORN". A key feature of manual control is that sending the same movement command twice acts as a toggle, stopping the robot.

- **Functionality**: This setup enables direct manual control of the robot's movements and functions, providing an alternative to the autonomous mode and facilitating testing or direct intervention.

Figure 4.9 shows an example user interface. The integration of BLE significantly enhances the robot's usability and interactivity.

Safety Feature - Manual Obstacle Block

The system implements a safety feature for manual forward movement. If the user commands the robot forward but the ultrasonic sensor detects an obstacle, the software flags the robot as blocked. It stops the motors and activates a buzzing sound to alert the user, preventing a collision. The user must then command a different direction (e.g., backward or turn) to clear the flag.

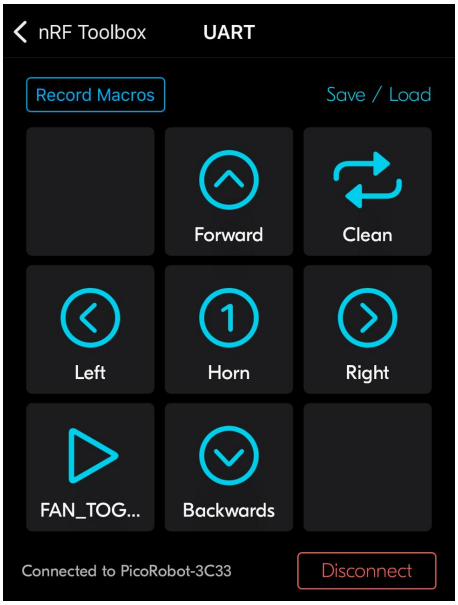


Figure 4.9: Example user interface of the NRF Toolbox application configured for controlling the Smart Cleaning Robot via BLE UART.

4.5.4 Software Deployment

The MicroPython firmware was flashed onto the Raspberry Pi Pico W. The control script (`main.py`) was uploaded using the Thonny IDE, as shown in Figure 4.10. A `boot.py` script was configured to enable WebREPL for wireless debugging and parameter tuning during development.



Figure 4.10: Screenshot of Thonny IDE showing the MicroPython control script deployed to the Raspberry Pi Pico W.

4.6 Performance Evaluation and Results

The evaluation focused on functional verification of core features and qualitative observation of the robot's behavior in a typical indoor test environment.

4.6.1 Functional Verification and Observations

- **Manual BLE Remote Control**: Functioned perfectly. The robot exhibited prompt and accurate responses to commands sent via the NRF Toolbox application, with good connection stability over several meters. The PID-assisted forward command was noticeably straighter than unassisted control.
- **Autonomous Navigation**: The integration of wheel encoders and a PID control system resulted in a significant improvement in navigational accuracy over the previous open-loop system. Straight-line movement was noticeably more stable, with the positional PID controller effectively mitigating much of the drift caused by motor and surface inconsistencies. While not perfectly precise in all scenarios, the robot could now execute its back-and-forth cleaning pattern more reliably. The dual-objective PID controller produced more consistent turn angles. Furthermore, the newly implemented stall detection and recovery feature was tested and found to be effective in simulated "stuck" conditions.
- **Vacuum System Performance**:
  - *Debris Collection:* The vacuum system, using the PS3 fan, collected light debris like small paper pieces and dust from hard floor surfaces.
  - *Limitations:* Performance was constrained by two main factors: the absence of a check valve, leading to occasional spillage of collected debris at the end of an operation, and the unavailability of brushes, which limited the robot's ability to dislodge embedded dirt.
- **Power System**: The 12V Li-ion battery pack [22] and LM2596S buck converter provided stable power, achieving an operational runtime exceeding 60 minutes on a full charge during continuous autonomous operation. The overall system current and power are approximately 2.2A and 26W respectively, at a nominal 12V. This estimate is based on the main power-consuming components: the PS3 fan (rated $\sim$1.3A at 12V), and the two DC motors, which can draw over 0.8A combined under operational load. The remaining electronics contribute a smaller fraction to the total consumption. Peak current during motor startup or simultaneous operation of all high-draw components could be higher.
- **User Interface (Button/LED)**: The physical start/stop button and status LED functioned as designed, providing correct user interaction and feedback.

Table 4.2 summarizes the key performance metrics observed during testing.

Table 4.2: Performance Test Results Summary.

Metric	**Result**	**Notes**
BLE Connection Range	$>$5 m	Stable indoor connection
Straight-Line Stability	Good	PID reduces drift significantly
Turn Angle Consistency	Improved	More reliable than open-loop
Stall Detection	Functional	Recovery sequence works
Battery Runtime	$>$60 min	Continuous operation
Debris Collection	Light debris	Paper, dust on hard floors
System Power	$\sim$26W	At 12V nominal

4.6.2 Summary

The performance evaluation indicates that the upgrade to a closed-loop, PID-based control system was a success. While not achieving perfect motion, it dramatically improved the precision and reliability of the robot's movement compared to the initial prototype. The implemented control system enabled the robot's autonomous behavior to be more systematic and, crucially, allowed for the integration of vital safety features like stall detection and recovery. The custom PCB and overall physical construction represent a solid prototyping effort.

The evaluation also confirms that achieving flawless path execution with the current DC motor and chassis setup remains a challenge. The vacuum system, while functional for light debris, also has clear limitations impacting its practical cleaning effectiveness. These findings are valuable insights that directly inform the recommendations for future work.

Chapter 5

Conclusion and Recommendations for Future Work

5.1 Conclusion

This thesis has successfully detailed the design, implementation, and qualitative performance evaluation of a prototype Smart Cleaning Robot. The project was undertaken with the aim of developing a cost-effective automated cleaning solution, leveraging the Raspberry Pi Pico W microcontroller, readily available sensors, and custom 3D-printed components. Key achievements of this project include:

- The development of a functional robot prototype capable of BLE-based manual remote control, featuring an autonomous navigation system upgraded to a hardware-aware, hybrid feed-forward and PID control strategy.
- A comprehensive design process encompassing mechanical design using SolidWorks, electrical system design with a custom PCB, and an iterative embedded software development process in MicroPython that explored multiple control theories.
- Successful integration of various hardware components, including the Raspberry Pi Pico W, HC-SR04 ultrasonic sensor, L298N motor driver, wheel encoders, and a repurposed PS3 fan for the vacuum system.
- The implementation of a robust physical prototype with a neatly assembled chassis and electronics, capable of demonstrating significantly improved, though not perfect, autonomous movement.
- Functional verification showing excellent remote-control capabilities, a basic ability to collect light debris, and the successful operation of safety features like stall detection.

5.1.1 Cost Analysis

A primary objective of this thesis was to demonstrate the feasibility of a cost-effective cleaning robot. Table 5.1 compares the prototype's total cost against representative com-

mercial alternatives.

Table 5.1: Cost Comparison: Prototype vs. Commercial Robots.

Robot	**Approx. Cost (USD)**	**Key Features**
This Prototype	$80 (3950 EGP)	BLE control, PID nav, vacuum
Basic Robot Vacuums	$100–200	Random nav, basic vacuum
Xiaomi X20 Plus	$300–500	LiDAR SLAM, auto-empty
iRobot Roomba 980	$500–900	vSLAM, smart features

The prototype achieves its core functionality at approximately **60–90% lower cost** than entry-level commercial robots. While it lacks advanced features like SLAM-based navigation, it provides a functional platform demonstrating that affordable home robotics is achievable with accessible components. Ultimately, this project effectively demonstrated the feasibility of creating a smart cleaning robot with a modest budget and accessible technology. The iterative design process, especially in refining the software control strategy, was crucial for overcoming hardware challenges and achieving a higher level of functionality. The resulting prototype serves as a valuable foundation for affordable home robotics, successfully balancing core functionality with cost-effectiveness and providing a clear pathway for the more sophisticated enhancements outlined in this chapter.

5.2 Limitations

Despite the progress made, the current prototype and project have several limitations that should be acknowledged:

- **Autonomous Navigation Precision**: While the implementation of wheel encoders and PID control represents a major advancement, achieving perfect navigational precision remains a primary limitation. Inherent non-linearities in the low-cost DC gear motors result in small, cumulative errors. This issue is compounded by the fact that the wheel encoders were integrated after the initial chassis was fabricated, leading to mounting points that are not perfectly symmetrical and contributing to small, cumulative errors. The PID system works to actively correct these errors, but it cannot eliminate them entirely.

- **Lack of High-Level Path Planning**: The current system uses its enhanced precision to better execute its reactive coverage strategy (moving forward and turning at obstacles or after recovering from stalls). However, it lacks advanced mapping and localization capabilities, such as those enabled by SLAM techniques. This absence restricts the robot to reactive strategies rather than enabling truly intelligent, map-based environmental coverage, such as the systematic Boustrophedon pattern discussed in Chapter 2.

- **Vacuum System Effectiveness**:

  - *No Check Valve:* The absence of a check valve in the Debris Collection Chamber can cause collected debris to spill out when the fan is turned off or the robot is jostled.

  - *No Brushes:* The lack of mechanical brushes (roller or side), due to the local unavailability of suitable off-the-shelf components, significantly reduces the robot's ability to dislodge dirt from carpets or effectively clean edges and corners.

- **Basic Power Management**: The current system uses a battery and buck converter but lacks advanced power management features such as battery level monitoring, low-battery warnings, or automatic return-to-charge functionality.

5.3 Lessons Learned

The development of this prototype provided valuable engineering insights beyond the technical outcomes:

- **Hardware-Software Interaction**: The brownout issue highlighted that software algorithms cannot compensate for hardware limitations. Understanding the electrical constraints of the system is essential before designing control strategies.
- **Iterative Design Value**: The evolution from open-loop to PID to hybrid control demonstrated the importance of iterative refinement. Each failure mode informed subsequent design decisions.
- **Component Sourcing Challenges**: Local unavailability of certain components (brushes, specific sensors) required creative alternatives. Future projects should include sourcing contingencies.
- **Early Encoder Integration**: Adding wheel encoders post-fabrication led to asymmetric mounting. Integrating precision sensors from the initial design phase would improve performance.
- **Documentation Throughout**: Maintaining detailed notes during debugging accelerated problem diagnosis and solution development.

5.4 Recommendations for Future Work

The current prototype serves as a strong foundation for numerous potential enhancements and future research directions, directly addressing the identified limitations:

1. **Improved Navigation and Sensing:**
    - **Upgrade Drive Motors (Stepper or High-Torque DC):** To overcome the inherent precision limitations of the current DC motors, future work should investigate replacing them with alternatives like stepper motors. Steppers offer open-loop control with a high degree of accuracy, which could eliminate path deviation issues. Furthermore, selecting motors with higher torque—whether they are steppers or simply more robust DC gear motors—is crucial. This would improve performance on thicker carpets and enhance the robot's capability to overcome small obstacles.

- **Refined Path Planning Logic**: The successful integration of wheel encoders provides reliable odometry, paving the way for more intelligent navigation. Future work could leverage this improved positional awareness to implement more systematic algorithms, such as a true wall-following routine for thorough edge and corner cleaning, which would be a logical next step beyond the current obstacle-avoidance pattern.
- **Explore SLAM**: As a more advanced step, consider upgrading the processing capabilities (e.g., to a Raspberry Pi 4 with ROS) to integrate a **LiDAR** sensor for 360-degree environmental mapping. This would allow the robot to build a precise 2D map and enable truly intelligent navigation, moving beyond simple obstacle avoidance to strategic, map-based cleaning, which would enable the implementation of efficient coverage algorithms like the Boustrophedon pattern [26].

2. **Enhanced Cleaning Mechanism:**

   - **Redesign Intake and Debris Chamber**: Design and incorporate a one-way check valve into the Debris Collection Chamber to prevent spillage.
   - **Add Mechanical Brushes**: Integrate a motorized roller brush to agitate dirt from carpets and hard floors, and side brushes to sweep debris from edges and corners into the vacuum's path.

3. **Comprehensive Power System Management:** Future work should integrate a more holistic power management system. This includes implementing battery voltage monitoring via an ADC pin on the Pico W to provide low-battery warnings or trigger a safe shutdown. Furthermore, the experience with motor stalls under high PID load highlighted the need for real-time voltage sag detection. An upgraded system could monitor for these sags and potentially use a more powerful battery pack or motor drivers with higher current capacity to reliably support aggressive, high-torque maneuvers.

By addressing these areas, the smart cleaning robot can evolve from the current functional prototype into a more autonomous, efficient, and practical cleaning assistant for household environments.

References

[1] H. Choset, "Coverage for robotics - A survey of recent results," *Annals of Mathematics and Artificial Intelligence*, vol. 31, pp. 113–126, 2001. doi:10.1023/A:1016639210559.

[2] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard, "Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age," *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309–1332, Dec. 2016. doi:10.1109/TRO.2016.2624754.

[3] MathWorks, "SLAM (Simultaneous Localization and Mapping)," *MathWorks.com*. [Online]. Available: https://www.mathworks.com/discovery/slam.html. [Accessed: Jun. 17, 2025].

[4] News Desk, "What is SLAM and How Does It Works?," *GeoSpatialNext.org*, Apr. 6, 2023. [Online]. Available: https://www.geospatialnext.org/what-is-slam-and-how-does-it-works/. [Accessed: Jun. 17, 2025].

[5] AerospaceControlsLab, "Controlling Self Driving Cars," *YouTube Video*, Oct 27, 2016. [Online]. Available: https://www.youtube.com/watch?v=4Y7zG48uHRo&ab_channel=AerospaceControlsLab. [Accessed: Jun 11, 2025].

[6] Process Control Explained, "Feed Forward vs PID Control — The Ultimate Analogy," *YouTube Video*, Sep 2, 2022. [Online]. Available: https://youtu.be/--swZTO9zQw?si=LqjCV2ri7Ovstz4G. [Accessed: Jun 17, 2025].

[7] Cytron Technologies, "HC-SR04 Ultrasonic Sensor Datasheet," [Online]. Available: https://u.pcloud.link/publink/show?code=XZbASX0Z9gMyDiAkjqhJzCEkpUJFMjk8Sbo7. [Accessed: Feb. 28, 2025].

[8] D. Nedelkovski, "Ultrasonic Sensor HC-SR04 and Arduino – Complete Guide," *HowToMechatronics.com*. [Online]. Available: https://howtomechatronics.com/tutorials/arduino/ultrasonic-sensor-hc-sr04/. [Accessed: Jun. 17, 2025].

[9] Raspberry Pi Ltd, "Raspberry Pi Pico W Datasheet," Jun. 30, 2022 (and updates). [Online]. Available: https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf. [Accessed: Feb. 28, 2025].

[10] Open Robotics, “ROS Wiki,” [Online]. Available: https://wiki.ros.org. [Accessed: Feb. 28, 2025].

[11] S. Agrawal, “The Ultimate Floor Cleaning Robot (Version 2.0) — How To Make,” *YouTube Video*, Oct 17, 2020. [Online]. Available: https://youtu.be/ecNfo2phP6g?si=qw_WiqSsPzh9q380. [Accessed: Feb. 28, 2025]. See also: https://github.com/saiyam-agrawal/Floor-Cleaning-Robot-v2, https://www.instructables.com/The-Ultimate-Floor-Cleaning-Robot-v20/.

[12] Rizwan’s Ideas, “How to Make Arduino based Smart Vacuum Cleaner Robot Best for Science Project,” *YouTube Video*, Jan 28, 2021. [Online]. Available: https://youtu.be/hHIIWpZcYNs?si=sa-vqxozIV0g641t. [Accessed: Feb. 28, 2025]. Code: https://drive.google.com/file/d/1XFB8C14F7sbg4Hh2A2rEGbN7wCO8UCnh/view. Website: https://rizwansideas.com/arduino-based-smart-vacuum-cleaner-robot/.

[13] SciNix, “How I Built My Own Self-Navigating Vacuum Cleaner,” *YouTube Video*, Jan 15, 2023. [Online]. Available: https://youtu.be/n8C8cyyUXuY?si=_KWLaZMLRyqeD7BE. [Accessed: Feb. 28, 2025]. Code: https://github.com/SKD-4/RVAC.

[14] Sabin Civil Engineering, “The Amazing Engineering behind Cleaning Robots!,” *YouTube Video*, Sep 1, 2022. [Online]. Available: https://youtu.be/hoY2YxLGV98?si=vKkoi0o1rWYs65PX. [Accessed: Feb. 28, 2025].

[15] iRobot, “iRobot Roomba 980 Product Page (Restored Model Example),” [Online]. Available: https://www.irobot.com/en_US/roomba-vacuuming-robot-irobot-roomba-restored-980/R980R99.html. [Accessed: Feb. 28, 2025].

[16] The Tech Chap, “Should You Buy a ROBOT Vacuum Cleaner? (Roomba 980 Review) — The Tech Chap,” *YouTube Video*, Aug 29, 2017. [Online]. Available: https://youtu.be/csGdz5_gZiY?si=ReL_dOrOgOdkM1wd. [Accessed: Feb. 28, 2025].

[17] Xiaomi, “Xiaomi Robot Vacuum X20 Plus Product Page,” [Online]. Available: https://www.mi.com/global/product/xiaomi-robot-vacuum-x20-plus/. [Accessed: Feb. 28, 2025].

[18] M. Wester, “Xiaomi Robot Vacuum X20+ Review – Unbeatable Value?,” *YouTube Video*, Jan 20, 2024. [Online]. Available: https://youtu.be/3XuFII1RM_M?si=f7JAoI4WfRL1rzRs. [Accessed: Feb. 28, 2025].

[19] Raspberry Pi Ltd, “Raspberry Pi Pico WH Product Page,” [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-pico/?variant=raspberry-pi-pico-wh. [Accessed: Feb. 28, 2025].

[20] STMicroelectronics, “L298 Dual Full-Bridge Driver Datasheet,” Rev 10, Sep 2013. [Online]. Available: https://www.st.com/resource/en/datasheet/l298.pdf. [Accessed: Feb. 28, 2025].

[21] Geeetech, “L298N Motor Driver Board,” Geeetech Wiki. [Online]. Available: https://www.geeetech.com/wiki/index.php/L298N_Motor_Driver_Board. [Accessed: Feb. 28, 2025].

[22] Generic Datasheet, “12V 4AH LITHIUM ION BATTERY PACK (3S2P),” [Online]. Available: https://u.pcloud.link/publink/show?code=XZ2CBk0ZCbvc2SPylpBn11XO9sDlc4GpDk1X. [Accessed: Feb. 28, 2025].

[23] PSDevWiki Contributors, “Fans - PS3 Developer Wiki,” Web Page, Section on Delta KSB1012HE. [Online]. Available: https://www.psdevwiki.com/ps3/Delta_KSB1012HE. [Accessed: Feb. 28, 2025].

[24] Texas Instruments, “LM2596 SIMPLE SWITCHER® Power Converter 150-kHz 3-A Step-Down Voltage Regulator,” SNVS124E, Revised May 2013. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm2596.pdf. [Accessed: Feb. 28, 2025].

[25] Raspberry Pi Ltd, “Raspberry Pi Pico-series Documentation,” [Online]. Available: https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html. [Accessed: Feb. 28, 2025].

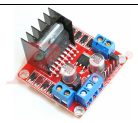
[26] H. Durrant-Whyte and T. Bailey, “Simultaneous Localization and Mapping: Part I,” *IEEE Robotics & Automation Magazine*, vol. 13, no. 2, pp. 99–110, Jun. 2006. doi:10.1109/MRA.2006.1638022.

Appendix A

Bill of Materials and Cost Analysis

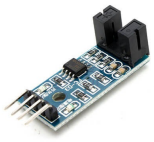
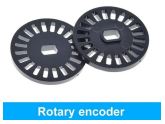
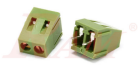
This appendix provides a comprehensive list of all mechanical and electrical components used in the construction of the prototype. The prices are listed in Egyptian Pounds (EGP) and reflect the market rates at the time of purchase.

Table A.1: Comprehensive Bill of Materials

Component	Photo	Qty.	Price/Unit (EGP)
Raspberry Pi Pico W (WH)		1	750.00
DC Motors (TT Mini Gearbox Pair)		2	50.00
H-Bridge L298N Motor Driver		1	80.00
HC-SR04 Ultrasonic Sensor		1	40.00
Battery Pack 12V 4000mAh		1	1400.00

Continued on next page

Table A.1 – *Continued from previous page*

Component	Photo	Qty.	Price/Unit (EGP)
PS3 Fan (Already Owned)		1	1000.00*
DC-DC Step Down LM2596S		1	50.00
IR Photoelectric Encoder Sensor		2	25.00
Rotary Encoder Disc		2	5.00
Push Button (DS-316 Green)		1	7.50
LED Indicator (XD10-3 Green)		1	7.00
2 Pin Screw Terminal		3	5.00
3 Pin Screw Terminal		6	6.00
Pin Header Female 1x20		2	3.00
Resistors (2.2kΩ, 4.7kΩ)		2	∼2.00

Continued on next page

Table A.1 – *Continued from previous page*

Component	Photo	Qty.	Price/Unit (EGP)
Jumper Wires (Assorted)		3 set	35.00
Speaker Wire (1m)		1	7.50
Robot Wheels (66mm)		2	36.00
Metal Caster Wheel		1	30.00
PCB Spacers (20mm)		8	0.75
Bolts and Nuts (M3)		4 bag	30.00
Heat Shrink Sleeve		2	10.00
		Subtotal (Purchased Components):	**2950.00**
		Subtotal (Owned/Estimated*):	**1000.00**
		Grand Total Estimated Cost:	**3950.00**